

**The University of Texas at Tyler
College of Engineering
Tyler, TX 75799**

**Final Design Report
For**

**Wearable Glove for American Sign Language (ASL) Gesture
Recognition**

**A design project to fulfill the requirements of Senior Design in the Department of Electrical
and Computer Engineering at The University of Texas at Tyler**

The individuals whose names and signatures appear below certify that the narrative, diagrams,
figures, tables, calculations, and analyses contained within this document are their original work
except as otherwise cited.

Jalen Bell


Signature

02/09/2026

Chase Brown


Signature

02/09/2026

Jared Carrig


Signature

02/09/2026

Acknowledgements

We would like to thank our advisor Dr. Frederic Rizk for his assistance, counsel, and supervision. He provided much needed assistance and demonstrated understanding and grace when providing feedback. We'd also like to thank UTTyler for providing laboratory space to develop and work. We'd also like to thank you for hiring such a great professor to aid us in our project.

Executive Summary

Sensors are an important part of many solutions in the modern technology landscape. Several different kinds of sensors are used for real-time systems, wireless systems, security, water management systems, and many other fields. This project, while a stark difference from other gesture recognition solutions, implements sensors in a wearable real-time system to aid the vocal and hearing impaired. Our project, “Wearable Glove for American Sign Language (ASL) Gesture Recognition,” is a solution to translating ASL for a larger audience. It implements several different sensor subsystems: an Inertial Motion Unit subsystem, a hall effect sensor subsystem, and a contact sensor subsystem to interact with a microcontroller subsystem which handles local, lightweight prediction model to make predictions based on the sensory data.

The prototype can serve as a teaching tool for the basics of ASL as it predicts the ASL alphabet and the first ten numbers. It can accurately distinguish the difference between “static” gestures (those that don’t require motion such as ‘A’, ‘R’, and ‘1’) and “dynamic” gestures (anything that requires motion such as ‘J’, ‘Z’, and ‘10’) thanks to the Inertial Motion Unit (IMU) which can distinguish motion of the wrist and determines orientation on the wrist for distinguishing similar gestures.

Two of the sensor subsystems make use of the I2C connections on the ESP32 in order to send in the sensory data through the serial data pin (A4) and implements the I2C serial clock pin (A5). The ESP32 allowed for easy programming implementation of the I2C connections as well as easy implementation of the Analog to Digital Converter (ADC) connections required for the hall effect sensor subsystem. The Arduino IDE provides libraries from Adafruit that make data collection from the IMU and ADC convenient.

The proposed solution is inexpensive to reproduce and implements the BLE capabilities provided from the Arduino ESP32 Nano (the chosen microcontroller). As mentioned previously, the ESP32 makes use of a lightweight, random forest model that is deployed to ensure proper, accurate predictions of gestures and to decrease throttling of the ESP32 and its dual core processors. The prototype is powered by a 3.7V 3000mAh Li-Po battery that uses a step-down buck converter to lower the voltage to 3.3V and power the prototype. BLE is used as a response to this constraint.

Gestures are predicted with above required accuracy and show appropriate accuracy for most gestures responses. Dynamic gestures are predicted with little issue as are more static gestures, but some similar gestures (such as ‘P’/‘Q’ and ‘F’/‘9’) cause confusion within the model.

Table of Contents

List of Figures.....	v
List of Tables.....	vi
Chapter 1 - Project Description.....	1
Chapter 2 - Design Specification.....	3
Chapter 3 - Design Alternatives and Final Design Solution.....	11
Chapter 3.1 - Design Alternatives.....	12
Chapter 3.1.1 - Top-Level System Architecture Selection.....	14
Chapter 3.1.2 - Subsystem Evaluation.....	17
Chapter 3.1.2.1 - Microcontroller Evaluation.....	17
Chapter 3.1.2.2 - Finger-Bend Sensor Evaluation.....	18
Chapter 3.1.2.3 - Inertial Motion Unit Evaluation.....	20
Chapter 3.1.2.4 - Contact Sensor Evaluation.....	21
Chapter 3.2 - Product Architecture and Subsystem Simulation.....	23
Chapter 3.3 - Hardware Subsystems.....	24
Chapter 3.3.1 - The Contact Sensor Subsystem.....	24
Chapter 3.3.2 - The Hall Effect (Finger Bend) Subsystem.....	26
Chapter 3.3.3 - The Inertial Motion Unit Subsystem.....	27
Chapter 3.4 - Software Architecture.....	28
Chapter 3.5 - Software Modules.....	30
Chapter 3.5.1 - Sensor Acquisition and Feature Processing Module.....	30
Chapter 3.5.2 - Random Forest Interface and BLE Communication Module.....	31
Chapter 3.6 - Hardware Off-The-Shelf Items.....	31
Chapter 3.7 - Software Off-The-Shelf Items.....	32
Chapter 3.8 - Schematics and Mechanical Drawings.....	33
Chapter 3.9 - Custom Software.....	33
Chapter 4 - Prototype Design and Fabrication.....	34
Chapter 4.1 - Materials Needed.....	34
Chapter 4.2 - Assembly Process.....	35
Chapter 4.3 - Work Plan.....	36
Chapter 5 - Testing and Validation.....	38
Chapter 5.1 - Testing Methodology.....	38

Chapter 5.2 - Experimental Results and Analysis.....	38
Chapter 5.3 - System Validation.....	46
Chapter 6 - Broader Impacts of the Project.....	50
Chapter 7 - Conclusions.....	51
Chapter 7.1 - Conclusion.....	51
Chapter 7.2 - Future Work.....	51
Chapter 7.3 - Final Remarks.....	52
Chapter 8 - References.....	53
Appendices.....	54
Appendix A - Work Breakdown Structure (WBS).....	54
Appendix B - User Manual.....	55
Appendix C - Knowledge Skill Table.....	62
Appendix D - Hardware and Features.....	63
Appendix E - Code.....	70
Appendix F - ASL Charts.....	94

List Of Figures

Figure 2.1 - General Hardware Layout.....	4
Figure 3.1.1 - Overview of Hardware Subsystems.....	11
Figure 3.2.1 - Hardware Block Diagram.....	23
Figure 3.3.1 - Contact Sensor Placement (Front of Hand).....	24
Figure 3.3.2 - Contact Sensor Placement (Back of Hand).....	25
Figure 3.3.3 - Contact Sensor Pad Subsystem Wiring Diagram.....	26
Figure 3.3.4 - Hall Effect Subsystem Wiring Diagram.....	27
Figure 3.3.5 - Inertial Motion Unit Wiring Subsystem Diagram.....	28
Figure 3.4.1 - Software Flowchart.....	30
Figure 5.2.1 - Hall Effect Trend (Raw).....	39
Figure 5.2.2 - Hall Effect Trend (Normalized).....	40
Figure 5.2.3 - Model Comparison.....	41
Figure 5.2.4 - Random Forest Confusion Matrix (Normalized).....	42
Figure 5.2.5 - Decision Tree Confusion Matrix (Normalized).....	43
Figure 5.3.1 - Output of Model (Arduino IDE).....	47
Figure 5.3.2 - Output of Model (BLE Output).....	48
Figure 5.3.3 - Final Prototype Design.....	49
Figure A.1 - Work Breakdown Structure.....	54
Figure A.2 - G.A.N.T.T. Chart.....	55
Figure D.1 - Full Design Schematic.....	63
Figure D.2 - Hall Effect Sensor Finger Housing Schematic.....	64
Figure D.3 - Hall Effect Sensor Finger Housing.....	65
Figure D.4 - Random Forest Prediction Confidence Histogram.....	66
Figure D.5 - Decision Tree Prediction Confidence Histogram.....	66
Figure D.6 - Random Forest Feature Importances.....	67
Figure D.7 - Decision Tree Feature Importances.....	67
Figure F.1 - ASL Alphabet.....	94
Figure F.2 - ASL Numbers 1-10.....	95

List of Tables

Table 2.1 - Materials Overview	3
Table 2.2 - Evaluation Basis for Initial Design.....	6
Table 2.3 - Engineering Standard Guidelines for Project.....	7
Table 2.4 - List of Constraints.....	8
Table 2.5 - Requirement Solution Traceability Matrix	9
Table 3.1.1 - Subsystem Weighted Decision Matrix.....	12
Table 3.1.2 - Top-Level Weighted Decision Matrix.....	15
Table 3.1.3 - Microcontroller Weighted Decision Matrix.....	17
Table 3.1.4 - Finger-Bend Sensor Weighted Decision Matrix.....	19
Table 3.1.5 - Inertial Motion Unit Weighted Decision Matrix.....	20
Table 3.1.6 - Contact Sensor Weighted Decision Matrix.....	21
Table 3.6.1 - Hardware Off-The-Shelf Items.....	31
Table 3.7.1 - Software Off-The-Shelf Items.....	32
Table 4.1.1 - Prototype Bill of Materials.....	34
Table 4.3.1 - Work Plan.....	36
Table 5.2.1 - Hall Effect Sensor Raw Output.....	39
Table 5.2.2 - Random Forest Summary.....	41
Table 5.2.3 - Gesture Statistics.....	43
Table 5.2.4 - Battery Life Test Results.....	45
Table 5.3.1 - Target Requirement Evaluations.....	46
Table C.1 - Knowledge Skill Table.....	62

Chapter 1 - Project Description

American Sign Language (ASL) transcription is a uniquely difficult problem as conventional audio-based automated transcription solutions do not work with visual mediums. Therefore, unique hardware must be designed in order to efficiently and effectively capture and record ASL gestures.

‘The Wearable Glove for ASL Gesture Recognition’ was designed with the function of translating each letter of the ASL alphabet along with numbers one through ten. The design involves the utilization of several sensors to record and monitor their position as well as communicating with the main microcontroller to log captured positioning data into a database that is called on for training. Sensors are needed for several things within the scope of this project in order to provide an effective solution:

- The bend of each finger
- Contact between the inside of middle and forefinger
- Contacts between the tips of the thumb, middle, and forefinger
- Contacts to palm
- Position/Movement of the wrist

Hall effect sensors are used to record measurements on the bend of each finger. One sensor across each metacarpal joint (first knuckle) is sufficient to measure the approximate bend of each finger. The Hall effect sensors were placed on the first knuckle of each finger and miniature magnets were placed along the phalanx to monitor the bend of the fingers when wearing the glove. The sensors are strong enough to notice magnetic force when fingers are stretched apart. This removes the need for sensors between fingers. This means that contact sensors aren’t needed and allows for other pins on the microcontroller for alternative use.

Contact sensor fabric serves as a solution to specific use cases such as making specific letters easier to recognize by the glove with higher accuracy. To handle issues with a letter such as ‘P’ or ‘Q’ in comparison to the number ‘6’ contact fabric was sewn to the fingertips of the middle finger and forefinger. Although ‘P’ and ‘Q’ are similar to ‘6’ and increase the likelihood of decreasing overall accuracy when translating. When the contact pad comes into contact with one of the fingers, it alleviates any confusion with what character the user is trying to make. In order to preserve translation accuracy, contact fabric between the thumb and forefinger helps identify when the letter ‘R’ is made. The forefinger and middle come into contact when making ‘R’, but it wasn’t able to be consistently accurate when identifying it. Contact fabric is also used on the palm in order to identify when the thumb comes into contact with the palm for additional accuracy when making certain characters.

An Internal Motion Unit (IMU) was used to record accurate wrist motion and position. While motion isn’t necessary for the scope of this project, recording accurate position of the hand and wrist allows for differentiation between similar characters such as ‘P’ and ‘Q’, ‘V’ and ‘2’, ‘C’ and ‘O’, etc.

Lastly, the Arduino ESP32 is used to handle the sensory input data and recognize what letters and numbers are being made. It uses existing libraries to communicate with the IMU to handle the complex wrist positions and motions. The microcontroller works in real-time with the sensors to provide accurate input and output readings. The ESP32 hosts a lightweight model that is trained with sensory data stored on a computer. This allows the glove to maintain portability and ensure low-computational effort when translating. A random forest was implemented due to its use of binary decisions when making predictions. The final, wearable product will function in real-time and translate each letter of the alphabet and numbers one through ten from ASL gestures into corresponding outputs.

Chapter 2 - Design Specifications

A total overview of necessary materials is as follows:

Category	Part	Qty	Specifications	Accuracy and Notes
Control Board	Arduino Nano ESP32	1	240 MHz dual-core, BLE + Wi-Fi, USB-C	~150 mA avg current
IMU	Adafruit BNO055 Absolute Orientation IMU	1	Small breakout (~25 × 20 mm)	Orientation ±3–5° stable; gyro ±3–5 % drift typical
External ADC	ADS1115 I ² C ADC	2	I ² C 4 ch each,	16-bit resolution
Bend Sensing	APS11760LUAA Linear Hall Sensors	5	3-pin analog (VCC/GND/Out)	~5–10 mV/V/T
Magnets	Neodymium Disc Magnet	5	3 × 2 mm	N/A
Contact Sensors	Conductive Fabric / Thread Pads	6	Thin (2–5 mm patches/strips)	N/A
Power	LiPo 3.7 V Battery	1	3000 mAh, 1.5A	~4–6 hours estimated life
	Buck Regulator	1	3.3 V, 1 A	Stable 3.3 V rail
Wearable	Cotton Glove	1	Medium	N/A

Table 2.1 - Materials Overview

The central processing will be done in an Arduino Nano ESP32 board. Input will mainly come from 5 linear Hall sensors, sent through an external analog-to-digital converter (ADC) channel for processing, contact points, and an IMU. The contact points and hall sensors provide information on finger bend and contact, while the IMU provides absolute orientation information, representing the location and orientation of the hand in space.

For our device we will need at least 3.3V supplied via the battery connection contacts on the back of the Arduino Nano ESP32. The battery will be 3000mAh, 1.5A, and 3.7V, which is more than we need and will allow for longer run times. Since the battery is 3.7V and the input needs to be 3.3V a buck converter will be used to assure the right voltage for operation. We lose

.5A to the buck converter so 1A is usable. The battery gives us around 6Wh of energy and the device will take around .61 Watts of energy per hour of operation, which brings us close to 8 hours of operation straight without needing to be recharged. For our use case 8 hours is plenty to gather measurements, and operate the device. A smaller capacity allows us to keep the weight of the device low and the size to a minimum. The device will also need to fit on the backside of a glove, for which we will use a medium cotton "jewelry inspection" glove for its snug fit and breathability.

The general connections between hardware components are represented by a flowchart. All of the parts mentioned above can be seen within the figure. The flowchart can be seen in figure 2.1 below:

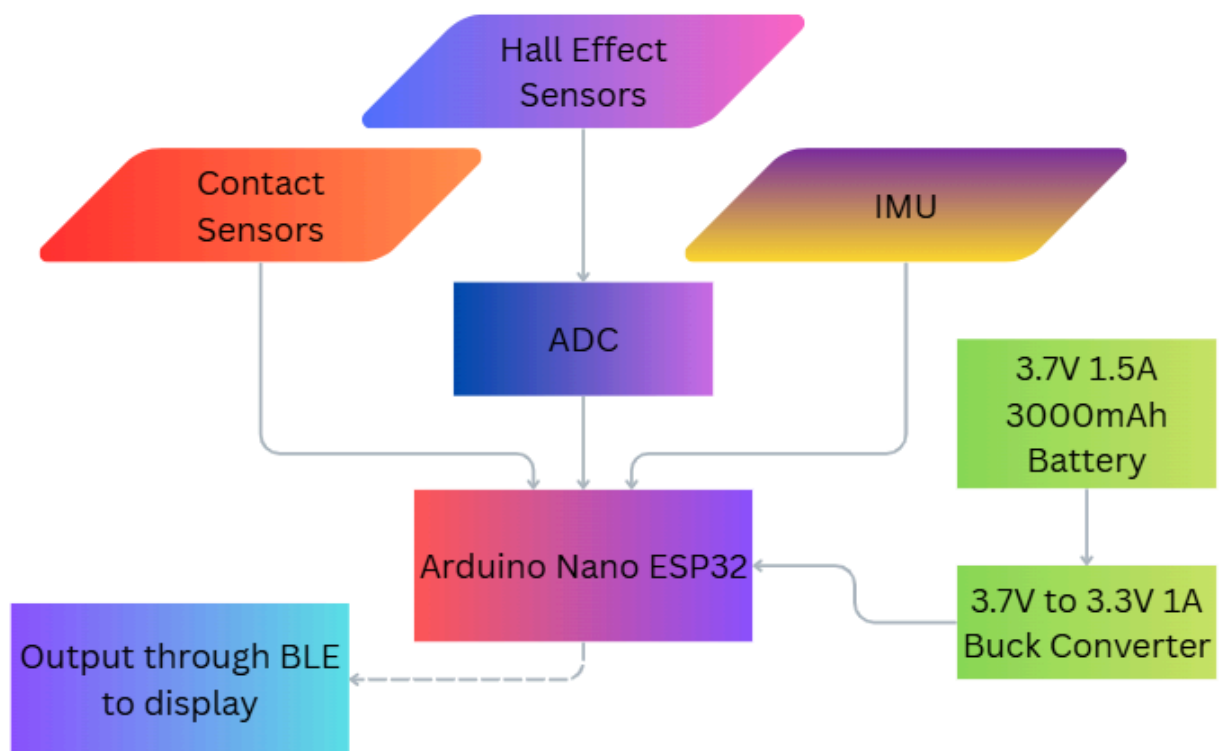


Figure 2.1 - Hardware Layout

The information provided in this setup will be stored on a PC and used to design the final model for ASL decoding. The model will be trained offline and uploaded to the ESP32 afterwards, at which point the only data being streamed from the glove to the PC will be predictions and confidence, which will then be displayed. This lightweight model will utilize a random forest and TinyML in order to perform with high accuracy when identifying positional data. TinyML is a less strenuous form of machine learning that is designed to run on smaller, low-powered embedded systems. It offers a solution to the limited memory of the ESP32 as well. A random forest makes use of binary branching paths (either yes or no) and allows the following features of the model to be distinguished from one another:

- Hall-effect sensors (Bend of fingers)
 - Thumb
 - Forefinger
 - Middle finger
 - Ring finger
 - Pinky finger
- Contact Fabric Sensors (Special cases)
 - Thumb
 - Between Middle and Forefinger
 - Forefinger
 - Middle finger
 - Palm
- IMU (Wrist motion)
 - Nine separate features for the nine-axis IMU

In total, nineteen different features are passed into the model to make predictions based on all of the sensory data received by the ESP32.

Ethical considerations are also a concern and need to be addressed. We considered the public health, safety and welfare, global factors, cultural factors, social factors, environmental factors, and economic factors. These were made in accordance with the *IEEE Codes of Ethics* and are shown in table 2.2:

Ethical Consideration	Evaluation Basis
Public Health	The product should pose no health risks when in use. Hardware is lightweight and easy to move. Insulation within the glove will protect the user from any sort of heat generated by the hardware. The glove itself weighs approximately 2.0g and possesses an approximate 4 to 6 hour use time.
Safety and Welfare	The product is safe and poses no risks. The product does promote the welfare of others with the ease of accurate and transportational glove that allows people to connect. The glove will be safe to wear for extensive periods of time thanks to insulation and efficient hardware placement. The product offers an affordable, assistive solution for communication.
Global Factors	The glove has little use outside of English speaking countries, but could prove useful in any ASL using countries such as Canada.
Cultural Factors	This provides English speaking countries the ability to connect easier with ASL users, but has smaller uses in other non-English speaking countries. Due to the limited scope of the project, the need for an ASL user will not be necessary as there exist many public documents of the ASL library that will be displayed during presentation. It will also serve as an accuracy check during testing. One of the members also has two years of ASL experience.
Social Factors	The project has the ability to influence the development of other sensory systems.
Environmental Factors	The model that is trained on the ESP32 is local and lowers ecological dangers compared to using a larger model. The battery can be recycled towards the end of product life.
Economic Factors	The hardware for the glove is inexpensive to produce. The glove could prove to be a profitable product.

Table 2.2 - Evaluation Basis for Design

All engineering projects must address the several engineering standards and codes that exist to streamline production, development, and ensure minimal issues with certification and cost. A table containing the engineering standards established by IEEE and other governmental and international standards that are followed during this project is shown below in table 2.3:

Organization	Engineering Standards
IEEE	● IEEE 802.15.4 (Wireless Personal Area Networks): Ensures reliable, low-power communication between sensors and microcontroller. ● IEEE 1472-2000 (Standard for Human Factors Engineering): Addresses user interaction with the wearable glove and ensures ergonomic usability.
Safety	● ANSI/ASSE Z10-2012 (Occupational Health and Safety Management Systems): Ensures the glove is safe for prolonged use without adverse health effects. ● IEC 61010-1 (Safety requirements for electrical equipment for measurement, control, and laboratory use): Ensures safety of electrical components used in the glove, like the microcontroller and sensors. ● IEC 62368-1 (Safety of Audio/Video, Information, and Communication Technology Equipment): Ensures the safety of electrical components in consumer electronic products like your wearable glove
Government	● FDA (Food and Drug Administration) Regulations: Ensures the safety and performance of wearable medical or assistive devices, if the glove is used in healthcare contexts. ● Americans with Disabilities Act (ADA): Ensures the glove is accessible and suitable for individuals with hearing impairments, as part of aiding communication.
International	● ISO 13485 (Medical Devices – Quality

	Management Systems): Ensures that the glove meets international standards for product design and user safety if the glove is marketed as a medical assistive device.
--	--

Table 2.3 - Engineering Standard Guidelines for Project

Several constraints must be addressed before considering larger design elements (such as power, speed, complexity, wearability, etc.). These constraints are shown below in table 2.4:

Constraint	Technical Description
Cost	The project was funded by the students, therefore finding inexpensive hardware was crucial to each of the group members. However, the hardware chosen should be able to produce accurate gesture reading. A balance between low cost and efficiency was made with the parts chosen for the final design. This also allowed for an easy to produce design that could be mass produced.
Power	The intended use of 3.3V to run the system imposed limitations regarding hardware. However, training and testing the system was conducted under 5V. The 3.3V applied from the battery to the Arduino ESP32 throttles the CPU and slows down the model. This required a battery with a current of 3000mAh or higher.
Speed	The model that was required to recognize gestures within 65 ms of the user making a gesture. The sensors would record data in real time and acceptable speeds, but the model would struggle to meet ideal speed conditions due to power constraints.
Complexity	The glove needed to maintain low physical complexity in order to manage connections to the microcontroller placed on the back side of the glove. It needed to maintain low software complexity in order to function quickly in read time.
Wearability	The glove had to meet the safety standards of the user according to the <i>IEEE Code of Ethics</i> . The glove had to maintain a low weight in order to allow ease of use for the user when wearing. Hardware needed to be placed

	and wired in such a way that it would not obstruct the users movements when making gestures. Battery placement posed a threat to safety and comfort.
Manufacturability	The glove maintained ease of manufacturability due to the economical size of the sensors (Hall effect, contact sensors, etc.), but wire management, microprocessor/battery placement, and insulation of the hand from hardware were all problems that we struggled against.

Table 2.4 - List of Constraints

This matrix demonstrates structured problem formulation and engineering reasoning by explicitly mapping system requirements to engineering challenges, design decisions, associated subsystems, and validation methods (Table 2.5):

System Requirement (Measurable)	Engineering Challenge	Design Decision	Subsystem(s) Involved	Validation Method
The system will achieve greater than 90% accuracy when classifying and identifying the ASL alphabet and numbers one through ten.	Similar gestures and patterns, calibration of the sensors, contact fabric issues, degradation of hardware	18-input vector (5 hall effect sensors, 5 contact sensors, and a 9-axis IMU), lightweight TinyML model, implementation of a 284-element engineered feature vector	Hall effect sensors subsystem (Analog to digital conversion), IMU subsystem (libraries used to convert IMU readings), ESP32 Random Forest subsystem (Random Forest deployed onto the embedded system)	Testing sample size includes 15 sample gestures for static gestures and 25 for dynamic gestures. There will be 15 sample gestures for numbers one through ten, compute and measure overall accuracy. Record the average correctness of each gesture
The system will have a maximum battery life of	ESP32 CPU throttling, conversion issues, ESP32	Lightweight TinyML model	Buck converter, ESP32 subsystem, Battery	Measure the current during operation, perform

four to six hours	draws power at a high rate, Power consumption of sensors		Subsystem	“runtime” tests
IEEE Ethics Practices and Standards/ Safety standards	Battery placement, insulation inside of glove	Place battery over a sleeve to be placed further down the arm (and to not obstruct gestures), two layers of gloves	Battery subsystem	Temperature measurements and physical inspection of contact surfaces
The model will be able to output a prediction (have a response time) of less than 65 ms	ESP32 CPU throttling, computational strain, difficult gestures	Use a lightweight model and use efficient programming to capture sensory data	Microcontroller subsystem	The program will record and measure latency as 65 ms is too quick to record by a human. A ‘C++’ program is used to record each response time. Times are then averaged.

Table 2.5 - Requirement Solution Traceability Matrix

Chapter 3 - Design Alternatives and Final Design Solution

A reliable, comfortable, and accurate solution that satisfied the system requirements and design goals previously discussed required several different subsystems and consideration of alternative hardware. We required the design to minimize power consumption for the 4 to 6 hour use time as well as providing a 90% accuracy rate when identifying gestures. This is achieved through the use of several subsystems as demonstrated previously in figure 2.1 and once again demonstrated in figure 3.1.1:

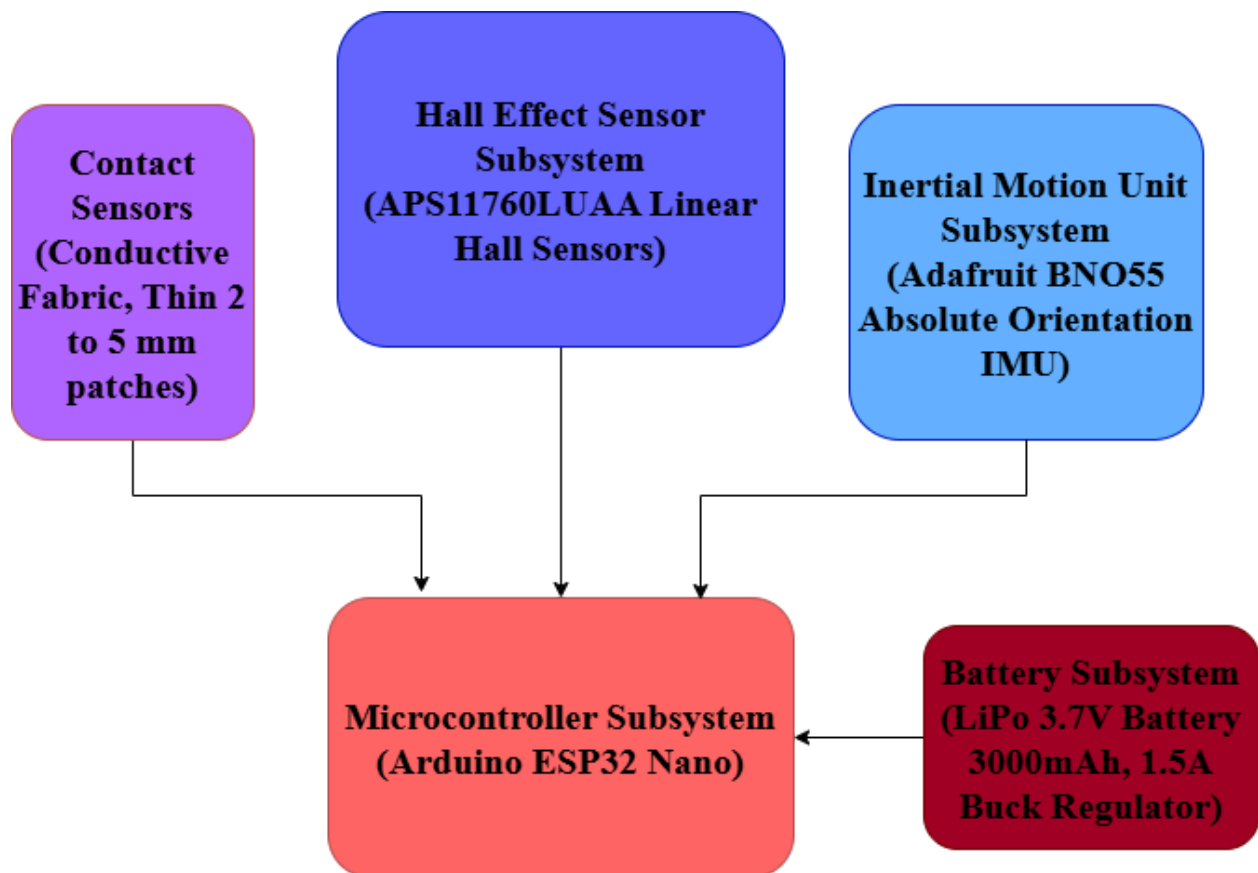


Figure 3.1.1 - Overview of Hardware Subsystems

The input sensor subsystem is made of three separate, smaller subsystems for finger-bend, thumb/finger contact, and wrist motion. The finger-bend sensor subsystem needs to measure the bend of the fingers and spacing between them (letters such as ‘X’ and ‘C’ concern finger bend and letters such as ‘F’ and ‘K’ concern finger spacing). The contact sensors are used for finding contact between the middle and forefinger (in the case of ‘R’) and for differentiating numbers and letters when counting (this concerns the gestures for ‘6’ to ‘10’). The wrist motion

subsystem allows for accurate positioning of the wrist (IMU helps identify ‘P’ and ‘Q’ from the standard use position).

Several alternatives were needed when considering the initial design and when assembling a prototype. Two different hall effect sensors were considered under power constraints along with several power alternatives. A subsystem decision matrix is showcased in 3.1 and showcases the many alternatives for each subsystem.

3.1 - Design Alternatives

A noticeable difference from other wearable ASL gesture recognition gloves is the use of hall effect sensors compared to the more commonly seen flex-based sensors. Some might be surprised to see the Arduino Nano ESP32 considering its popularity with Internet-of-Things solutions, sleep/wake patterns, and how it fairs against other competitors. The IMU was chosen for its compatibility with the microcontroller chosen with libraries meant specifically for it. Cost plays a major factor in all of the hardware implemented in the final design.

Several constraints applied to the hardware chosen for the final design. For example, hall effect sensors compared to flex-based sensors was a fairly simple decision based on degradation of the hardware and the lifetime of the product. Flex-based sensors would achieve the accurate readings needed to have the lightweight model recognize finger position, but further research demonstrated that flex-based sensors degrade quickly. The alternative was a hall-effect sensor that used magnets to determine the bend of a finger. Hall effect sensors degrade slower and would help with determining distance between metacarpal joints (knuckles) when fingers stretch apart or pull close together. Hardware had to face the constraints and factors of cost, efficiency, manufacturability, accessibility, and compactness. Decisions were made considering these factors with a scale of five points (5 being the best and 1 being the worst) used to judge how each piece of hardware fairs. These decisions are shown below in the Subsystem Decision Matrix (Table 3.1.1):

	Cost	Efficiency	Manufacturability	Accessibility	Compactness	Total Score
Finger Bend Input Sensors						
APS11760L UAA Linear Hall Sensors	5	4	5	4	5	23

APS11700 Vertical and Planar Hall Sensors	4	4	4	3	5	20
Bend Labs 1-Axis Flexible Flex Sensor	4	4	3	3	4	20
Encoded Inputs (Subsystem Concept)	3	2	3	3	2	13
Inertial Motion Unit						
Adafruit BNO055 Absolute Orientation IMU	4	4	3	3	4	20
LSM6DS33 3D Accelerome ter and Gyro Carrier	4	3	3	4	5	19
Contact Pad Sensors						
Conductive 'Smart' Fabric		4	3	3	5	
Metal Pads		4	3	2	4	
Microcontroller						
Arduino Nano ESP32	5	4	5	3	5	22

ESP32 C5 DevKitC1	5	4	3	2	4	18
Analog to Digital Converter (ADC)						
ADS1115 I²C ADC	4	4	4	5	5	22

Table 3.1.1 - Subsystem Decision Matrix

A rubric was created to evaluate hardware and systems based on their parts and their adherence to factors and ethical concerns. Three design alternatives are scored and judged based on the ethical factors and project constraints as previously discussed in Chapter 2. The results are shown in Table 3.1.2 (Ethical/Constraint Subsystem Decision Matrix):

3.1.1 - Top-Level System Architecture Selection

As shown in the Ethical/Constraint Subsystem Decision Matrix (Table 3.1.2) three different designs were developed in consideration with ethical and project constraints. The Top-Level Designs include:

- **Design 1:** Wearable and Portable Glove with BLE Applications (Battery Powered with On-Board Processing)
- **Design 2:** Wearable Glove with Wired Connection (On-Board Processing with External Computing Device)
- **Design 3:** Wearable and Portable Glove with Dedicated Website (Battery Powered with Wi-Fi and On-Board Processing)
- **Design 4:** Wearable and Portable Glove with Build-In Display (Battery Powered with Translation Display on Wrist)

Each alternative was evaluated according to the same ethical considerations outlined in Chapter 2 and by the IEEE standards. A weighted decision matrix reflects the relative importance of each category based on project restraints for cost, power, speed, complexity, wearability, and manufacturability (again, outlined previously in Chapter 2). Several different systems are concerned within this section including the Top-Level Designs and the categories that are found in Table 3.1.1 (The Subsystem Decision Matrix). Each weight sums to 1.00 within the following tables. The designs are scored out of five points with five being the best within a certain ethical category and one being the worst.

Evaluation Basis	Description of Relevance	Weight	Alt. Design #1 (BLE)	Alt. Design #2 (Wired)	Alt. Design #3 (Website)	Alt. Design #4 (Built-In Display)
Public Health	Should pose no threat to the skin health of the user wearing the glove (Take precaution with insulation due to batter placement). Thermal threats to skin health are the main concern.	0.15	3	5 (No battery)	3	3 (Extra concern of display)
Safety and Welfare	The glove should promote general welfare of the users through its use as a learning tool, translation device, or other usage. It should not restrict user movement and poses no security threats. The glove should stay portable to allow users to translate at will.	0.20	5	2 (Wired)	4 (Connect over Wi-Fi)	5
Global Factors	Easily applicable uses in other ASL speaking countries.	0.10	4	3	4	5
Cultural Factors	Must not pose any offense to ASL users. Should act as a supportive device to ASL users and translators.	0.20	4	4	3 (Website)	3 (English only display)
Social Factors	Glove should allow user	0.10	4	4	3 (Not as clear)	4

	human-technology communication to provide clear, efficient communication.					
Environmental Factors	Recyclable hardware is implemented to provide an environmentally sound solution to end-of-life concerns. Minimal infrastructure of the microcontroller.	0.15	4	4	3	4
Economic Factors	Hardware should be inexpensive	0.10	4	3	3	3
TOTAL SCORE	-	1.00	4.05	3.55	3.50	3.95

Table 3.1.2 - Top-Level Weighted Decision Matrix

(BLE)

Alt. Design 1: $(0.15*3) + (0.20*5) + (0.10*4) + (0.20*4) + (0.10*4) + (0.15*4) + (0.10*4)$

- **Total** = 4.05

(Wired)

Alt. Design 2: $(0.15*5) + (0.20*2) + (0.10*3) + (0.20*4) + (0.10*4) + (0.15*4) + (0.10*3)$

- **Total** = 3.55

(Website)

Alt. Design 3: $(0.15*3) + (0.20*4) + (0.10*4) + (0.20*4) + (0.10*3) + (0.15*3) + (0.10*3)$

- **Total** = 3.50

(Display)

Alt. Design 4: $(0.15*3) + (0.20*5) + (0.10*5) + (0.20*3) + (0.10*4) + (0.15*3) + (0.10*3)$

- **Total** = 3.95

The weighted evaluation table shows the Wearable and Portable Glove with BLE Applications maximizes safety of the user, provides clear predictions of gestures, support for end-of-life disposal, minimizes cost, and supports ASL communities. The evaluation also shows that the Wearable and Portable Glove with Build-In Display maximizes safety of the user, applicability in ASL speaking countries, minimizes cost, but doesn't support ASL communities as well as design one. This is due to the BLE applications of the first design. The BLE

applications can easily be reworked into an app that can translate into alternative languages while a display limits the translation capabilities. However, design four has the closest score to the first design. If some limitations or constraints damage the infrastructure of design one, then design four offers a healthy alternative.

3.1.2 - Subsystem Evaluation

Within each subsystem, alternative hardware is proposed in the following weighted matrices. The following weighted decision matrix demonstrates the evaluation of alternative hardware based on the constraints outlined in Table 2.4 in Chapter 2. Cost, power, speed, complexity, wearability, and manufacturability are taken into consideration, but may not be scored. Several tables use separate evaluation categories better relating to the subsystem.

3.1.2.1 - Microcontroller Evaluation

The microcontroller is an extremely mission critical subsystem when evaluating. The alternative options for the microcontroller and followed by the Microcontroller Weighted Decision Matrix (Table 3.1.4):

- **Alt. Microcontroller 1** - Arduino Nano 32
- **Alt. Microcontroller 2** - ESP32 C5 DevKitC1
- **Alt. Microcontroller 3** - ESP32 PICO V3

Evaluation Basis	Description of Relevance	Weight	Arduino Nano 32	ESP32 C5 DevKitC 1	ESP32 PICO V3
Cost	The microcontroller should minimize the overall cost of the design.	0.20	4	3	5
Power	The microcontroller should consume minimal power to maximize battery life	0.15	4	4	5
Speed	The microcontroller should be responsive	0.15	4	5	4

	enough to provide result in about 100ms				
Complexity	The microcontroller should allow for simple programming and retraining	0.20	5	3	2
Compactness	It is necessary for the microcontroller to fit comfortably on the wrist	0.20	3	2	5
Accessibility	The microcontroller should be accessible for assembly	0.10	5	4	2
Weighted Total	-	1.00	4.10	3.35	3.95

Table 3.1.3 - Microcontroller Weighted Decision Matrix

(Arduino Nano ESP32)

Alt. Design 1: $(0.20*4) + (0.15*4) + (0.15*4) + (0.20*5) + (0.20*3) + (0.10*5)$

- **Total** = 4.10

(ESP32 C5 DevKitC1)

Alt. Design 2: $(0.20*3) + (0.15*4) + (0.15*5) + (0.20*3) + (0.20*2) + (0.10*4)$

- **Total** = 3.35

(ESP32 PICO V3)

Alt. Design 3: $(0.20*5) + (0.15*5) + (0.15*4) + (0.20*2) + (0.20*5) + (0.10*2)$

- **Total** = 3.95

The Arduino Uno succeeds in its ease of accessibility. It is implemented easily with the selected IMU and has excellent computing speed. The ESP32 PICO provides a decent alternative that could provide better power consumption and calculation speed. However, it greatly increases the complexity of the system which would result in a large decrease in accessibility.

3.1.2.2 - Finger-Bend Sensor Evaluation

Unique and specific sensors are necessary in order to monitor the bend of fingers to ensure maximum accuracy when having the model recognize ASL gestures. The alternative sensors are shown in the Finger-Bend Sensor Weighted Decision Matrix (Table 3.1.5):

- **Alt. Finger-Bend Sensor 1** - APS11760LUAA Vertical Hall Sensors

- **Alt. Finger-Bend Sensor 2** - APS11700 Vertical and Planar Hall Sensors
- **Alt. Finger-Bend Sensor 3** - Bend Labs 1-Axis Flexible Flex Sensor

Evaluation Basis	Description of Relevance	Weight	APS11760LUAA Vertical Hall Sensors	APS11700 Vertical and Planar Hall Sensors	Bend Labs 1-Axis Flexible Flex Sensor
Cost	Bulk purchases of sensors should maintain a low cost	0.15	5	5	3
Power	Sensors should not interfere with battery power constraints	0.10	3	4	3
Accuracy	Sensors should provide accurate readings of finger bend	0.20	4	4	4
Compactness	Sensors should limit size due to compact area of the glove	0.20	5	5	4
Durability	Sensors should have extended life duration	0.20	5	4	1
Accessibility	Sensors should be easy to access	0.15	4	4	3
Weighted Total	-	1.00	4.45	4.35	3.00

Table 3.1.4 - Finger-Bend Sensor Weighted Decision Matrix

(APS11760LUAA Vertical Hall Sensors)

Alt. Design 1: $(0.15*5) + (0.10*3) + (0.20*4) + (0.20*5) + (0.20*5) + (0.15*4)$

- **Total** = 4.45

(APS11700 Vertical and Planar Hall Sensors)

Alt. Design 2: $(0.15*5) + (0.10*4) + (0.20*4) + (0.20*5) + (0.20*4) + (0.15*4)$

- **Total** = 4.35

(Bend Labs 1-Axis Flexible Flex Sensor)

Alt. Design 3: $(0.15*3) + (0.10*3) + (0.20*4) + (0.20*4) + (0.20*1) + (0.15*3)$

- **Total = 3.00**

Hall Effect sensors provide improvements to more common designs implementing Flexible Sensors as they last longer in comparison.

3.1.2.3 - Inertial Motion Unit Evaluation

The IMU proves necessary insurance of accuracy by providing the model with necessary positional data. This helps differentiate gestures such as “T” and “2” which are otherwise the same excluding the position of the wrist. The choices are evaluated within the Inertial Motion Unit Weighted Decision Matrix (Table 3.1.6):

- **Alt. IMU 1** - Adafruit BNO055 Absolute Orientation IMU
- **Alt. IMU 2** - LSM6DS33 3D Accelerometer and Gyro Carrier

Evaluation Basis	Description of Relevance	Weight	Adafruit BNO055 Absolute Orientation IMU	LSM6DS33 3D Accelerometer and Gyro Carrier
Cost	The cost of the IMU is not as important as other factors, as accuracy and redundancy take precedence.	0.10	4	5
Power	IMU should stay compatible with previous power specifications (3.3V)	0.20	5	5
Compatibility	Can the IMU implement easily with software (i.e. through libraries)	0.25	5	3
Compactness	Does the IMU prohibit movement due to size or space?	0.20	4	4
Redundancy	Does the IMU provide greater accuracy with more axes?	0.25	5	3 (Only three)

Weighted Total	-	1.00	4.70	3.80
-----------------------	---	------	------	------

Table 3.1.5 - Inertial Motion Unit Weighted Decision Matrix

(Adafruit BNO055 Absolute Orientation IMU)

Alt. Design 1: $(0.10 \times 4) + (0.20 \times 5) + (0.25 \times 5) + (0.20 \times 4) + (0.25 \times 5)$

- **Total** = 4.70

(LSM6DS33 3D Accelerometer and Gyro Carrier)

Alt. Design 2: $(0.10 \times 5) + (0.20 \times 5) + (0.25 \times 3) + (0.20 \times 4) + (0.25 \times 3)$

- **Total** = 3.80

The Adafruit Absolute Orientation IMU provides greater accuracy through the abundance of axes that it provides. The compatible libraries for the IMU that allow for easy software implementation with the Arduino Nano ESP32 allow for ease of implementation.

3.1.2.4 - Contact Sensor Evaluation

The material implemented in the contact sensors would determine how well the glove would accurately detect contact between fingers (Forefinger and middle finger) and contact between fingertips (i.e. when counting numbers). The decisions are shown in the Contact Sensor Weighted Decision Matrix (Table 3.1.7):

- **Alt. Contact Fabric 1** - Contact Fabric
- **Alt. Contact Fabric 2** - Capacitive Glove
- **Alt. Contact Fabric 3** - Aluminum Foil

Evaluation Basis	Description of Relevance	Weight	Conductive	Capacitive Glove	Foil
Cost	A lower cost is preferred to keep overall cost down.	0.15	3	4	5
Compatibility	Material should be easily implementable within the system and mount to fabric easily	0.25	4	4	2
Compactness	Material should maintain a sleek	0.15	5	5	4

	design.				
Durability	Material should not degrade too quickly, nor should material be sensitive.	0.20	5	4	3
Accessibilty	Material should be readily available	0.25	4	4	5
Weighted Total	-	1.00	3.95	4.15	3.70

Table 3.1.6 - Contact Sensor Weighted Decision Matrix

(Conductive Fabric)

Alt. Design 1: $(0.15*3) + (0.25*4) + (0.15*5) + (0.20*5) + (0.25*4)$

- **Total** = 4.20

(Capacitive Glove)

Alt. Design 2: $(0.15*4) + (0.25*4) + (0.15*5) + (0.20*4) + (0.25*4)$

- **Total** = 4.15

(Aluminum Foil)

Alt. Design 3: $(0.15*5) + (0.25*2) + (0.15*4) + (0.20*3) + (0.25*5)$

- **Total** = 3.70

Conductive fabric, despite being harder to procure, is best due to its incredibly reliable nature for contact detections.

3.2 - Product Architecture and Subsystem Simulation

The following demonstrates several System Block Diagrams that illustrate the connections between hardware level components and a separate block diagram for software level components. It connects each of the subsystems previously seen in the Overview of Hardware Subsystems (Figure 3.1.1). Below is a Hardware-Level Block Diagram showcasing the interaction between parts (Figure 3.2.1):

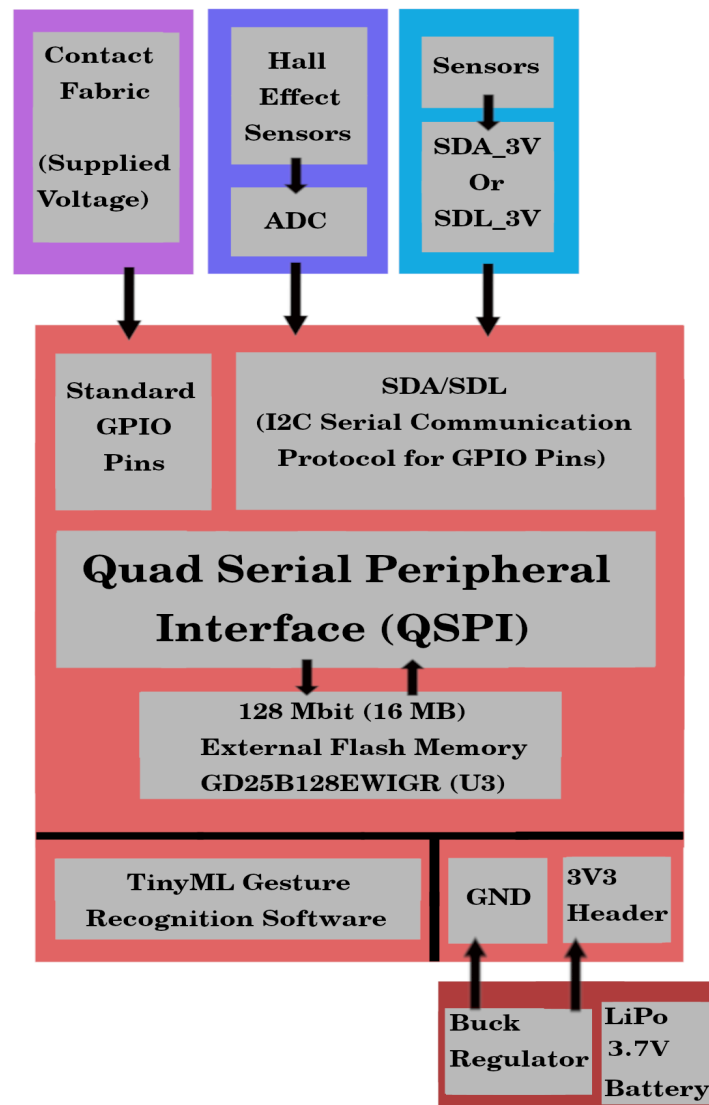


Figure 3.2.1 - Hardware Block Diagram

The colors of Figure 3.2.1 match with the previous diagram of Figure 3.1.1 where the purple block demonstrates the contact sensor subsystem (used for specific finger contacts), the dark blue demonstrates the hall effect sensor subsystem, the light blue demonstrates the IMU subsystem, the lighter red demonstrates the microcontroller subsystem, and the dark red demonstrates the battery/power subsystem.

3.3 - Hardware Subsystems

Within this section, the subsystems are discussed further in depth with diagrams to illustrate placement, wire connections, and other details regarding each subsystem discussed in figure 3.2.1 from the last section.

3.3.1 - The Contact Sensor Subsystem

This subsystem is simple. The fabric (placed according to figures 3.3.1 and 3.3.2) has each fabric sensor sewn into place with a voltage current and a connection to one of the several GPIO pins on the Arduino ESP32. This subsystem has a total of six connections to the microcontroller as can be seen in the diagram of where the contact pads will be placed:

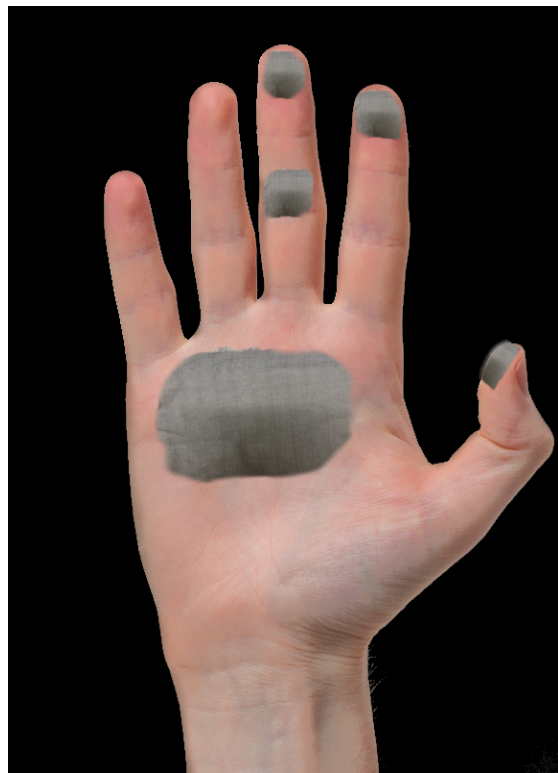


Figure 3.3.1 - Contact Sensor Placement (Front of Hand)



Figure 3.3.2 - Contact Sensor Placement (Back of Hand)

Please note that there only exist a total of six sensors. Figure 3.3.1 shows each of the sensors. The sensor on the side of the forefinger is supposed to be the same as the uppermost sensor in figure 3.3.2, shown above. The two sensors between the forefinger and middle finger serve to increase accuracy when identifying gestures such as “R”. Figure 3.3.2 also shows the thumb contact, but the palm and fingertip sensors are hidden within this figure. A wiring diagram of the contact sensors is shown below in figure 3.3.3 (Contact Sensor Pad Wiring Diagram):

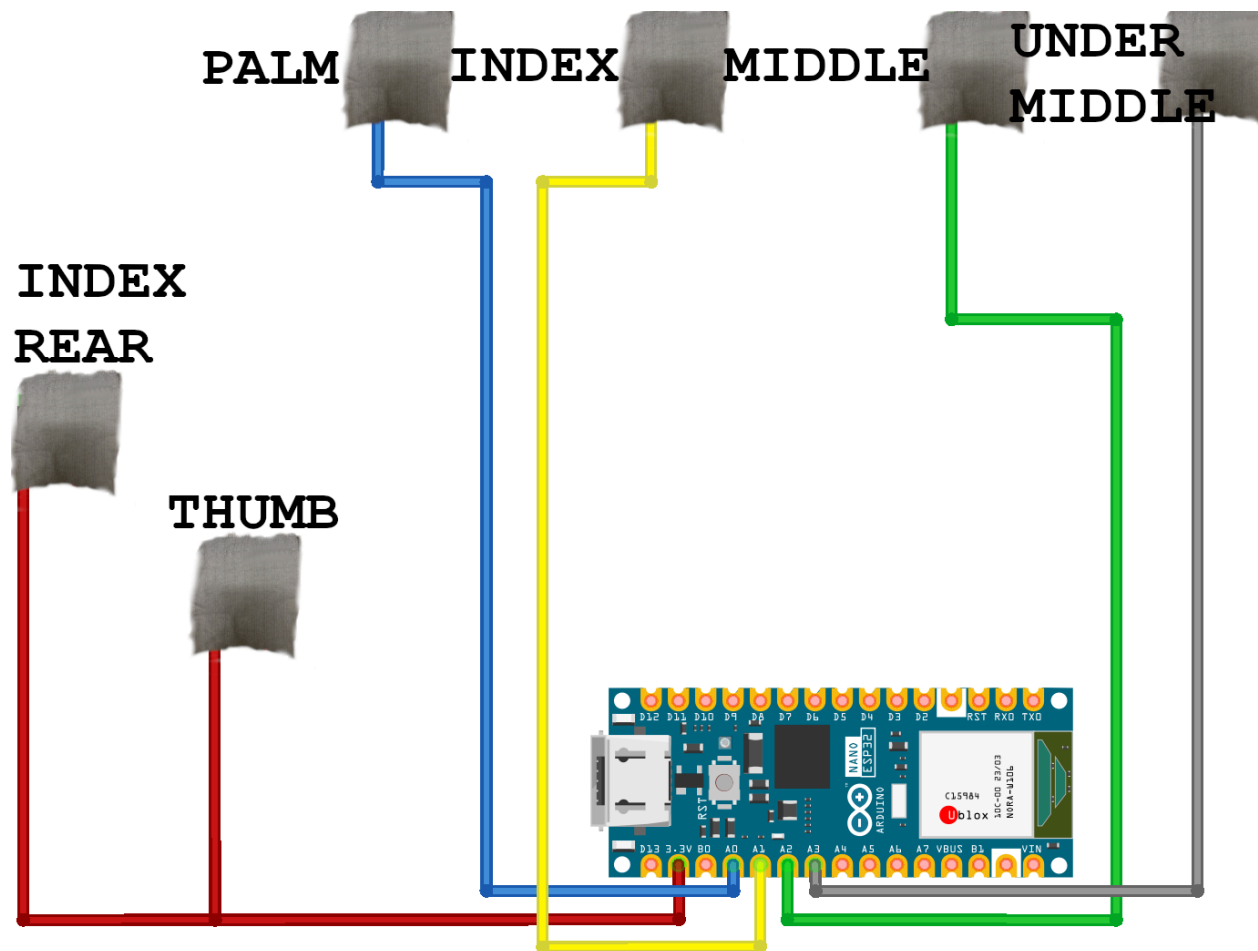


Figure 3.3.3 - Contact Sensor Pad Subsystem Wiring Diagram

The contact pad connected to the 3.3V pin of the microcontroller is sewn onto the thumb, so that when the forefinger, middle finger, ring finger, or palm come into contact with the thumb a circuit is completed. The GPIO pins receive the voltage readings and send the sensory data to the model for processing.

3.3.2 - The Hall Effect (Finger Bend) Subsystem

The hall effect sensors must connect to the Analog-to-Digital-Converter (ADC) in order to output the information through the SDA/SDL connection on the Arduino Uno ESP32. Further clarification can be seen in 3.9 Schematics and Mechanical Drawings. The wiring diagram shown in Figure 3.3.4 illustrates the connection between the hall effect sensors, their ADCs, and their connection to the microcontroller:

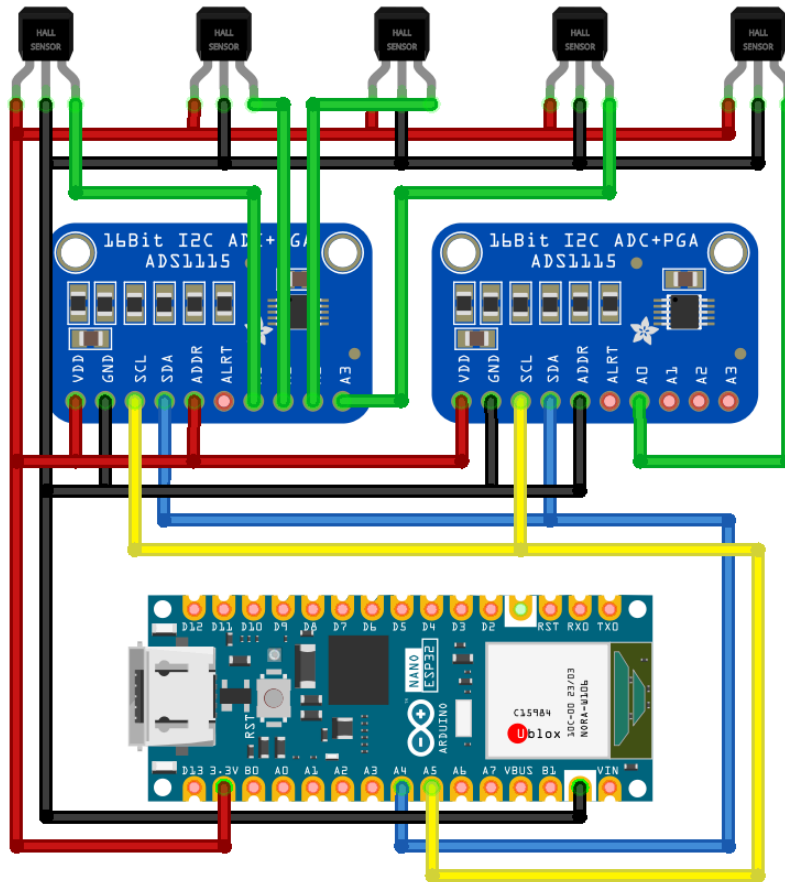


Figure 3.3.4 - Hall Effect Subsystem Wiring Diagram

3.3.3 - The Inertial Motion Unit Subsystem

The IMU has its own subsystem and simply needs to be wired to the microcontroller. Its subsystems are shown below in figure 3.3.5:

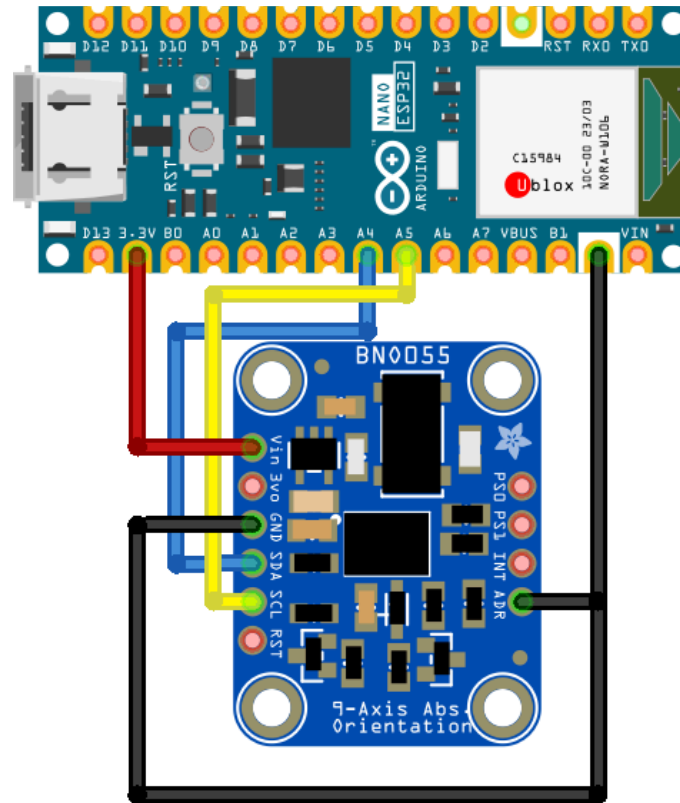


Figure 3.3.5 - Inertial Motion Unit Wiring Subsystem Diagram

3.4 - Software Architecture

For the software architecture portion of this design there are 4 major sections of code that are vital to the operation of the device. The first is the data collection software, which is vital in the running of the random forest model and the translation of American Sign Language. Sensor data is read in from the 5 APS11760LUAA hall effect sensors, inertial motion unit and the contact sensors.

The second is the random forest itself, which is vital in the data processing of the input data from all of the sensors. The model will be trained on a separate computer using tiny ML in python, then converted into C++ for smooth operation on the Arduino Nano ESP32. Even though the ESP32 has the capabilities to run some python, it is much less computationally intensive for the microcontroller. This leads to smoother, faster operation with the added benefit of giving the device longer battery life.

The third is the communication of the results of the random forest model to the user device. Once all data has been collected and processed a readable display is required for reading the results. Wireless communication alleviates the need for a wire to constantly be attached to the device and user device. Bluetooth Low Energy was chosen as the means of information communication for its low power usage and widespread adaptability among many softwares and

devices. This contributes to a stable clean connection to the user device to ensure data is sent cleanly.

The fourth is the user device software which will be responsible for taking in the results of the random forest model sent over BLE to display the results and metrics. This portion will be coded in python to take advantage of libraries such as tkinter for graphical user interfaces. When the data is received it will display what sign the device has predicted with a confidence score of how sure it is of its answer. Then a real photo of the sign will be displayed alongside the information to show what the sign is expected to look like for comparison to the user of the translation device. After the next sign is signed the device will again display the results to the user of what the sign was predicted to be, its confidence score and a real photo example of what the sign should look like.

Figure 3.4.1 (Software Flowchart) demonstrates the software architecture described above. It makes use of several if statements to check for proper initialization. BLE initialization is not crucial and does not cause a fatal error. The final loop begins where sensory data is taken in and a prediction is made.

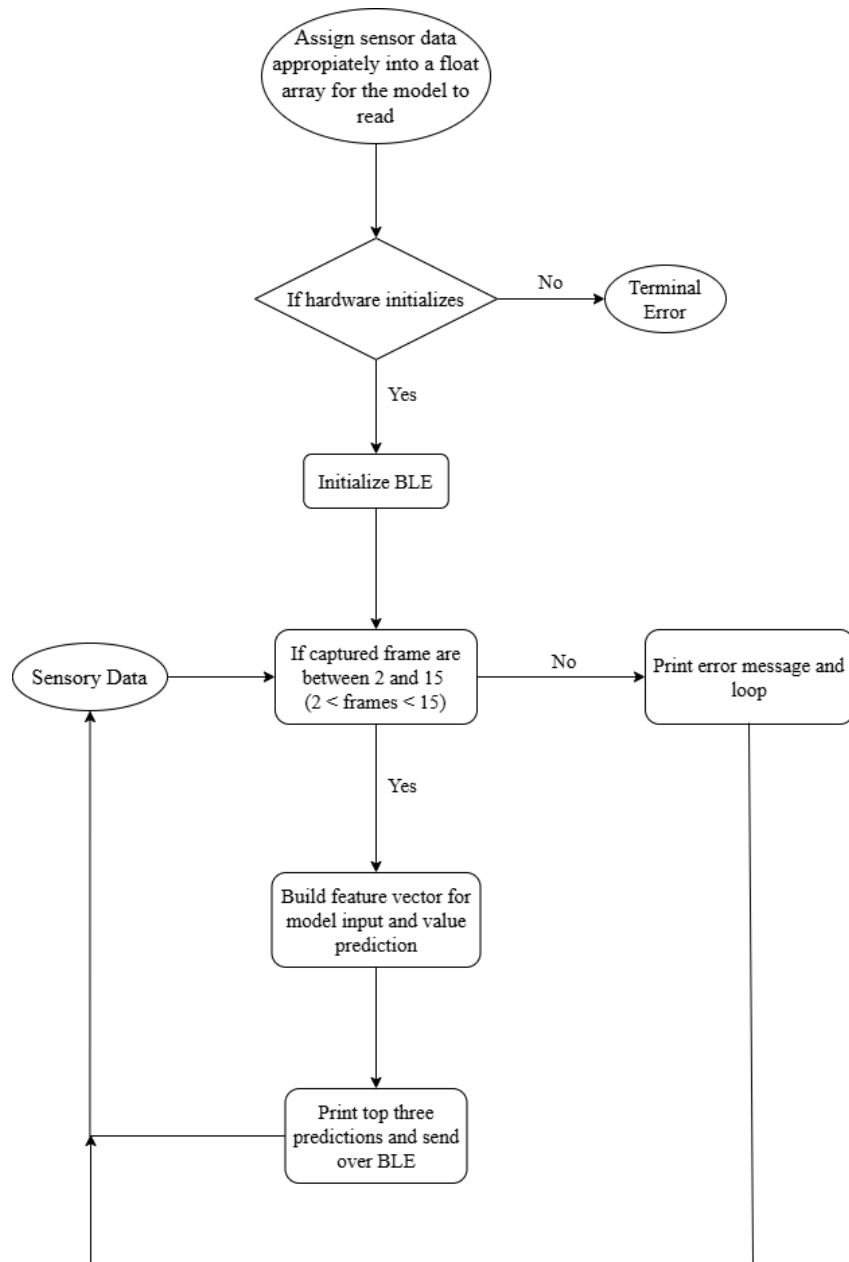


Figure 3.4.1 - Software Flowchart

3.5 - Software Modules

3.5.1 - Sensor Acquisition and Feature Processing Module

This module is responsible for collection and preparing all sensory inputs for classification. It reads the five hall effect sensors through the ADS1115 ADC, digital contact

sensors through GPIO pins, and nine-axis orientation data from the BNO055 IMU. The raw data is filtered (moving average / debounce), normalized to match training conditions, and organized into a 18-element feature vector.

This module ensures consistent real-time sampling and proper formatting of sensor data before it is passed to the machine learning subsystem. Proper signal conditioning is critical to maintaining the system's target classification accuracy (>90%) as defined in the design requirements.

3.5.2 - Random Forest Interface and BLE Communication Module

This module performs real-time ASL gesture classification using a lightweight random forest model trained offline and deployed to the ESP32 using TinyML. Each gesture sequence is converted into 284 engineered features explained in appendix D. These features summarize the entire motion (mean, max, change, sampled points, etc.) across all 18 sensor channels, along with timing and magnitude statistics. Therefore, rather than predicting from a single frame, the model makes decisions based on a full sequence summary, so the Arduino must compute the same features before running the model. Reference Appendix D.8 for an explanation of each type of feature.

After interfacing, the predicted class and confidence level are transmitted via Bluetooth Low Energy (BLE) to an external display device. The lightweight structure of the random forest ensures prediction occurs within the system speed constraint (<65ms) while minimizing computational load and power consumption.

3.6 - Hardware Off-The-Shelf Items

The following items were used in the construction and development of the design with the name, part number and datasheet URL of each part bought.

Part Name	Part Number	Datasheet URL
APS11760LUAA-0PL	620-2014-ND	https://www.allegromicro.com/~media/Files/Datasheets/APS11700-60-Data-sheet.ashx
Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout - BNO055	2472	https://cdn-learn.adafruit.com/assets/assets/000/125/776/original/bst-bno055-ds000.pdf?1698865246

MakerFocus 2pcs 3.7V 3000mAh Lithium Polymer Rechargeable Battery	6680133	https://www.fpbattery.com/wp-content/uploads/2024/06/fpbattery-3093106-3.7V-3000mAh-Lithium-Polymer-Battery-Specification.pdf
Pololu 3.3V, 1A Step-Down Voltage Regulator	D24V10F3	https://www.pololu.com/file/0J842/isl85410-datasheet.pdf
ADS1115 16 Bit 16 Byte 4 Channel I2C IIC Analog-to-Digital ADC PGA Converter	ADS1115	https://www.ti.com/lit/ds/symlink/ads1115.pdf
Arduino Nano ESP32	ABX00092	https://docs.arduino.cc/resources/datasheets/ABX00083-datasheet.pdf

Table 3.6.1 - Hardware Off-The-Shelf Items

3.7 - Software Off-The-Shelf Items

The following software packages were used in the construction and development of the design with the name of the software and a link to the website

Name	Link
Adafruit_BNO055 by Adafruit	https://github.com/adafruit/Adafruit_BNO055
Adafruit_Sensor by Adafruit	https://github.com/adafruit/Adafruit_Sensor
EasyBNO055_ESP	https://github.com/madhephaestus/EasyBNO055_ESP
ESP32 BLE Arduino	https://github.com/espressif/arduino-esp32/tree/master/libraries/BLE
Random Forest ML	https://docs.arduino.cc/libraries/tinydecisiontreeclassifier/?utm_source=chatgpt.com
numpy	https://github.com/numpy/numpy
scikit-learn	https://github.com/scikit-learn/scikit-learn
micromlgen	https://github.com/eloquentarduino/micromlgen
pandas	https://github.com/pandas-dev/pandas

matplotlib lib	https://github.com/matplotlib/matplotlib
joblib	https://github.com/joblib/joblib
bleak	https://github.com/hbldh/bleak
tkinter	https://github.com/python/cpython
Arduino IDE by Arduino	https://docs.arduino.cc/software/ide/
Visual Studio Code by Microsoft	https://code.visualstudio.com
KiCad	https://www.kicad.org/download/

Table 3.7.1 - Software Off-The-Shelf Items

3.8 - Schematics and Mechanical Drawings

Please see Appendix D for a list of schematics and drawings.

3.9 - Custom Software

The code snippets that were used in the development and operation of the American Sign Language translation device can be found in Appendix E.

Chapter 4 - Prototype Design and Fabrication

The final prototype is a glove with all necessary materials attached on the back of the hand and the wrist. This includes five linear 3.3V Hall-effect sensors, five neodymium magnets, an ESP32 microcontroller, ADS1115 analog-to-digital converters, and a wrist-mounted electronics module.

4.1 - Materials Needed

The following table lists components for the assembly of this prototype, along with hyperlinks for purchase of each significant electronic component.

Component	Manufacturer	Part Number	Qty	Unit Price (\$)	Total Price (\$)
Microcontroller	Arduino	ABX00092 (Nano ESP32)	1	18.30	18.30
ADC IC (Breakout)	Qoroos	ADS1115	2	14.95	29.90
Hall Sensor	Allegro Microsystems	APS11760LUAA-0PL	5	0.97	4.85
IMU	Adafruit	BNO055	1	34.95	34.95
LiPo Battery	MakerFocus	3.7V 3000mAh Protected	1	15.00	15.00
3.3V Buck Converter	Polulu	D24V10F3	1	8.00	8.00
Conductive Fabric	Adafruit	20cmx20cm	1	14.95	14.95
Neodymium Magnet	Generic	3×8 mm Disc	5	0.13	0.65
Cotton Glove	Generic	Medium Cotton Glove	1	5.00	5.00
Wrist Strap	Generic	Nylon Webbing	1	5.32	5.32
TPU Filament	Generic	Any	N/A	0.10	0.10
				Total	136.99

Table 4.1.1 - Prototype Bill of Materials

Most non-electronic parts for this design can be purchased locally, or even reused if already owned.

4.2 - Assembly Process

First, gather all necessary materials and ensure the glove fits snugly on the hand. It is important that all dimensions are known before assembly, as small distances can cause variation in the output of the hall effect sensors.

The hall effect sensors and magnets will be mounted across the knuckles of each finger using the CAD model found in Appendix D. It is important that flexible filament is used to ensure consistent bend and longer lifetime performance. For these purposes, TPU was selected. These mounts have holes to fit 8x2 magnets and APS11760 LUA hall effect sensors. These can simply be glued in. It is important to attach the wires to the pins and insulate the connections before the final placement of the hall effect sensors in order to prevent shorts between pins. The mounts can then be glued to the glove across each knuckle with the hall effect sensor on the proximal side and positioned so that its sensing face is toward the magnet.

Next, the IMU needs to be mounted on the back of the hand to record the facing, acceleration, position, and movement of the hand. To do this a patch of non-stretch cotton fabric, larger than the IMU, must first be sewn on the back of the palm. This will prevent fabric deformation from influencing the interpretations.

The contact points simply need to be attached as shown in Figure 3.3.1. If using contact fabric, small 1cm by 1cm squares will be plenty, except for the palm, which should cover the whole of the palm. Wires must be attached and connected according to Figure 3.3.3. A simple whip stitch will suffice for attachment purposes.

Moving on to the wristband, mount the electronics on the wristband or some wrist carrier to improve comfort and to reduce motion at the finger sensors. For this a 1 inch wide strip of nylon webbing was used. The mounting holes were used to sew the electronics directly to the nylon webbing for prototyping, but it is recommended to use a chassis or backing plate for reproduction purposes.

The 3.7V LiPo battery must be stepped down to 3.3V to be usable. A buck converter is used for this, simply connecting the JST red and black to the buck IN+ and IN-, respectively. After this, the OUT+ of the buck becomes the 3.3V rail and OUT- the ground, which all electronics will share. The battery should be attached to the underside of the wrist on the nylon webbing or wristband used. The attachment of the battery is important and it needs to be secure and insulated. The battery can be detachable, and this is likely the safest route for charging, as an integrated battery chassis was not completed.

The analog digital converters are, along with the 3.3V and ground lines, the only pieces connected to the hall effect sensors. The output of each hall effect sensor will be input into one of the two ADCs. The thumb will get a reserved ADC chip, and the index, middle, ring, and pinky finger will get input 0, 1, 2, and 3 of the second ADC, respectively.

The Arduino now must connect to all the pieces as shown in the wiring diagrams in Appendix D. Wires should be cut with enough slack to keep connections stress-free throughout the full range of motion.

Finally, utilizing the code in Appendix E, capture each gesture a minimum of 50 times each to create enough labeled data to train your preferred model, or simply utilize the model provided here. Appendix E also contains the link to the code to stream the model over BLE to a laptop running Arduino software and display responses.

4.3 - Work Plan

Table 4.3.1 describes the time to realize the project, both with original times for project completion and estimated reproduction times.

Step #	Task Description	Detailed Activities	Design & Development Time (hrs)	Reproduction Time (hrs)
1	Requirements & Architecture Planning	Define feature vector, subsystem boundaries, TinyML selection, layout planning	10	3
2	Component Research & Selection	Evaluate sensors, MCU, IMU, battery sizing	12	2
3	Power Budget & Electrical Analysis	Current draw estimation, buck converter sizing	6	1
4	Glove Layout Planning	Sensor placement and magnet alignment planning	8	2
5	Hall Sensor Mounting	Attach 5 Hall sensors across MCP joints	2	1
6	Magnet Installation	Install 3x2mm neodymium magnets on fingers	4	2

7	Contact Fabric Installation	Sew fingertip, palm, and inter-finger contact pads	4	2
8	ADC & I2C Wiring	Wire ADS1115 boards and verify signal integrity	6	3
9	IMU Integration	Mount IMU and verify communication	5	2
10	ESP32 Integration	Connect all sensors and power rails	6	3
11	Power System Assembly	Integrate LiPo battery and buck converter	5	2
12	Firmware Development	Sensor reading and feature extraction firmware	15	4
13	Feature Vector Construction	Normalize and format 18-input vector	6	2
14	Model Training	Collect dataset and train random forest	18	6
15	TinyML Deployment	Convert and flash model to ESP32	6	2
16	System Debugging	Resolve noise, alignment, and threshold issues	20	5
17	Accuracy Testing	Collect test samples and compute confusion matrix	10	4
		Total	143	43

Table 4.3.1 - Work Plan

Chapter 5 – Testing and Validation

5.1 - Testing Methodology

The glove's accuracy was verified first by individual function and then by translation accuracy. This was done through verifying the model's predictions on labeled datasets in which each static gesture had at least 15 different examples for data (static gestures being any in which the hand does not make major movements and sit still). Each dynamic gesture had at least 25 different examples to ensure accuracy (dynamic gestures are any that require the user to move their hand such as 'Z', 'J', and the number '10'). Testing was completed with real-world usage where each gesture (letters A–Z and numbers 1–10) was performed each ten times with the accuracy output by a simple python program. These recorded accuracies from each test were then used to generate a representative dataset (Table 5.2.3 - Accuracy By Gesture). The gestures were tested in order of the alphabet and then by each number in order to ensure each gesture is able to be predicted accurately. This was the testing methodology to check if the project met the first set of system requirements established in Table 2.5 - Requirement Solution Traceability Matrix.

The battery life was tested during the testing of the glove's accuracy. Testing the battery was as simple as leaving the battery strapped to the testing user for an extended period of time to check for both safety of battery heat against the skin as well as battery life.

Finally, when discussing the response time of the glove (or the output latency) the response time is the amount of time it takes for the model to output its prediction beginning with the end of the reading time (the reading time being the measurement of time in which the glove stops recording) to the moment the model outputs its prediction. The time can be thought of as:

$$\text{Response Time} = (\text{Time of Output}) - (\text{End of Reading Time})$$

The system requirements specify that Response Time should be less than 65 milliseconds. The total amount of time it takes from the beginning of recording the sensory data to the time model outputs the prediction is a separate measurement.

5.2 - Experimental Results and Analysis

The outputs of the IMU and contact points of the glove were simple to verify through unmodified serial outputs from the Arduino ESP32. The hall effect sensors required more than this to validate, as magnetic field strength is not a sufficient measure of function. In order to address this, measurements were recorded at multiple approximate finger positions:

	Full Extension	One Third Bend	Two Thirds Bend	Full Bend
Thumb	4600	7660	7920	8490
Index	4414	6887	7750	8220
Middle	2258	5850	6943	8070
Ring	3080	5380	7000	7900
Pinky	2772	6400	7600	8220

Table 5.2.1 - Hall Effect Sensor Raw Output

As we can see, these values are quite disparate due to external factors such as magnetic interference, though they do result in a recognizable trend:

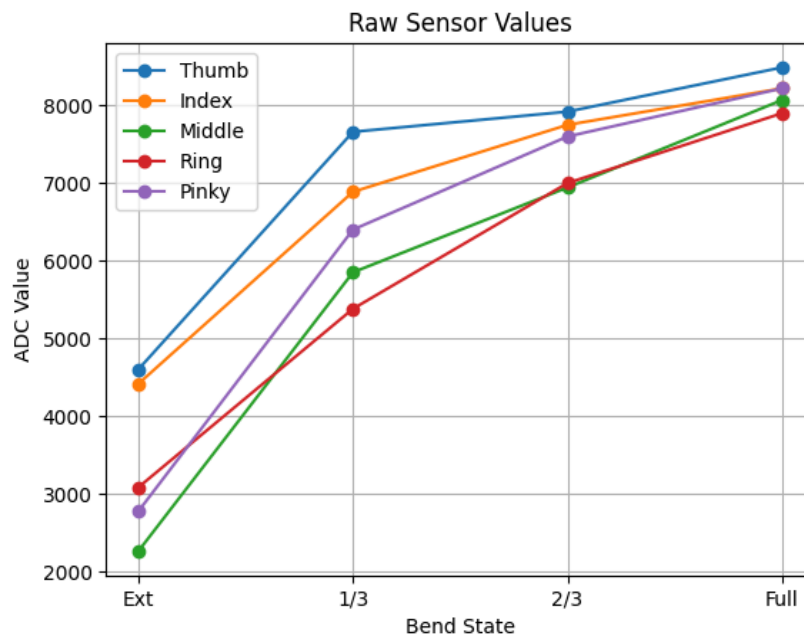


Figure 5.2.1 - Hall Effect Trend (Raw)

When normalized, the values show a much more predictable and consistent pattern:

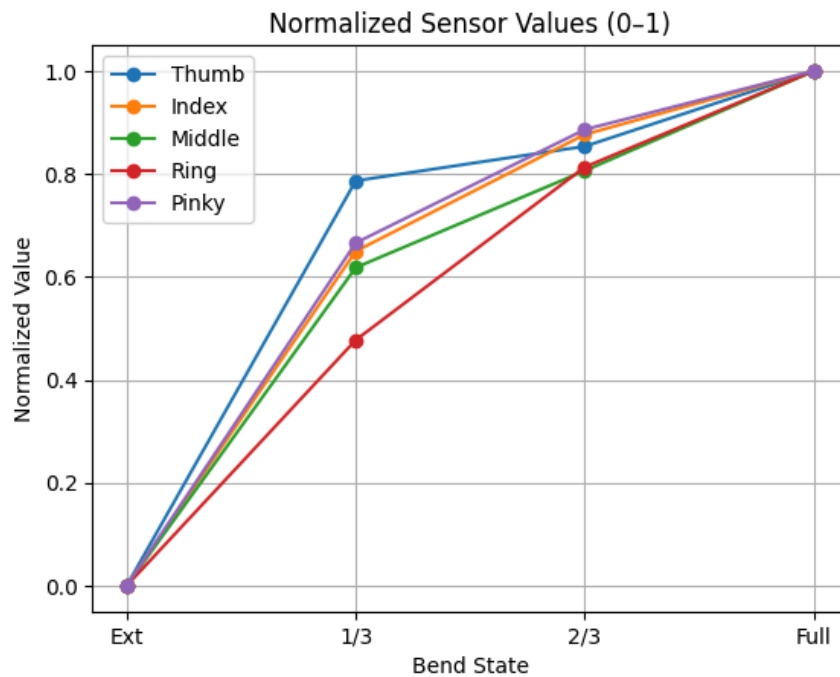


Figure 5.2.2 - Hall Effect Trend (Normalized)

Once the function was verified, input data was gathered, recorded, and labeled by gesture. Two models were trained off of the input data, with one using a random forest model and the other a decision tree model. Each used the same preprocessed input consisting of 284 total input features including statistics like average, min/max, total movement, rate of change, and sampled values across time explained in appendix D. It also includes overall gesture properties (duration, number of frames) and combined IMU magnitudes to capture the intensity and shape of the motion.

The Overall performance of both models are shown in Figure 5.2.3:

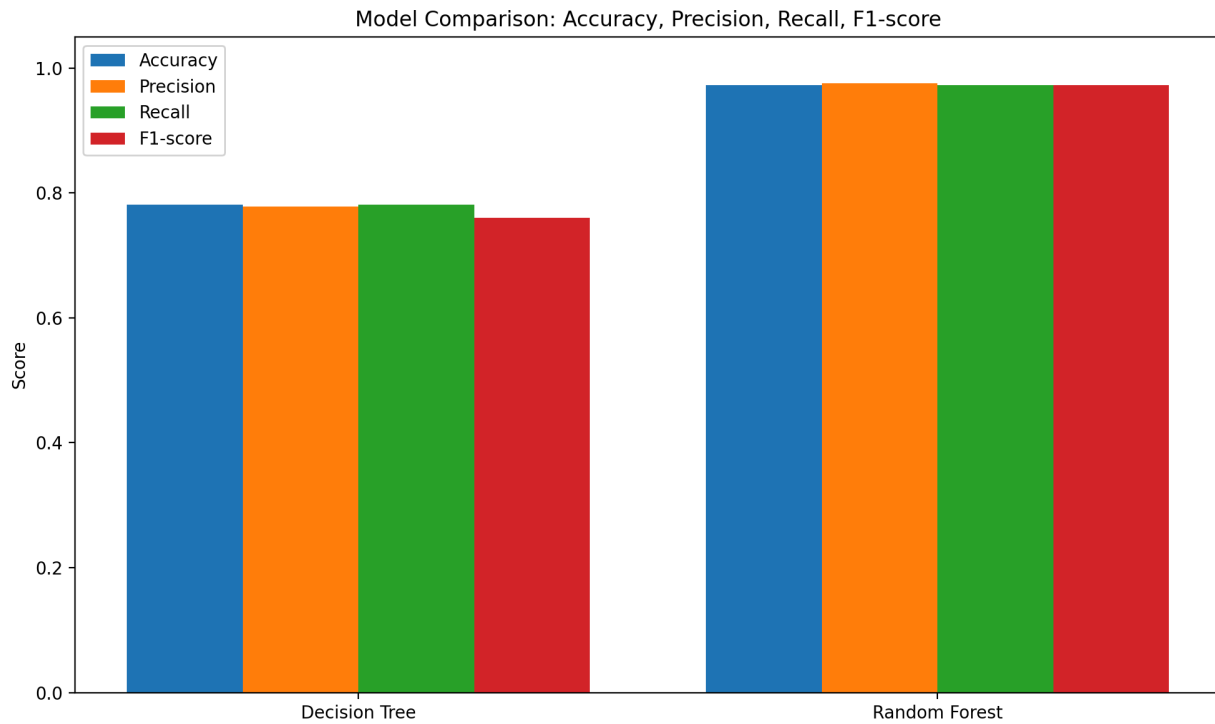


Figure 5.2.3 - Model Comparison

Random Forest Summary Table	Value
Accuracy	.9723
Precision	.9752
Recall	.9723
F1-score	.9723

Table 5.2.2 - Random Forest Summary

The results indicate that the system achieves reliable classification performance across most gestures. Higher accuracy was observed in gestures with distinct finger configurations, while lower accuracy occurred in gestures with similar hand shapes which was as to be expected. The random forest model also performed notably better, with accuracy (97.23%), precision (97.52%), and recall (97.23%) all above 90% when measured after training, so it was chosen to

be deployed on the glove and utilized for further testing. Details of each model can be found in Appendix B.

Pictured in figure 5.2.4 is the confusion matrix for the random forest, which is the model that was chosen to deploy onto the microcontroller:

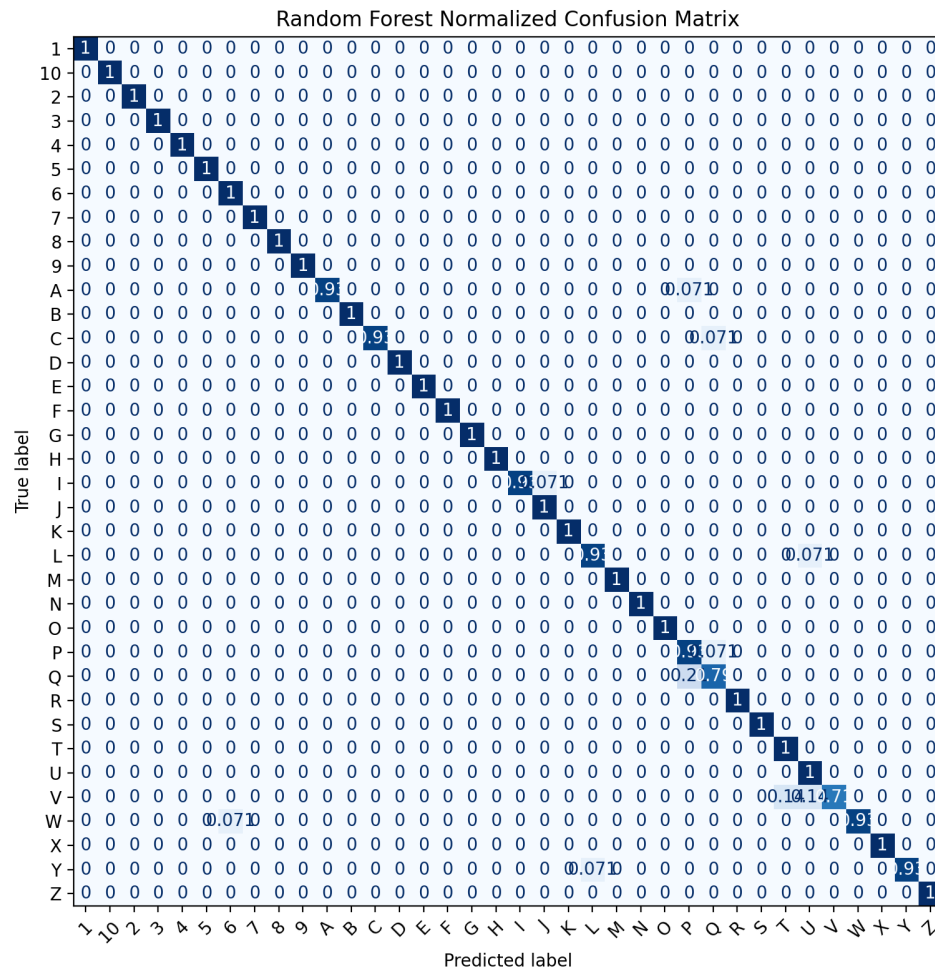


Figure 5.2.4 - Random Forest Confusion Matrix (Normalized)

The initial results of the Decision Tree Model that was not chosen are shown in Figure 5.2.5:

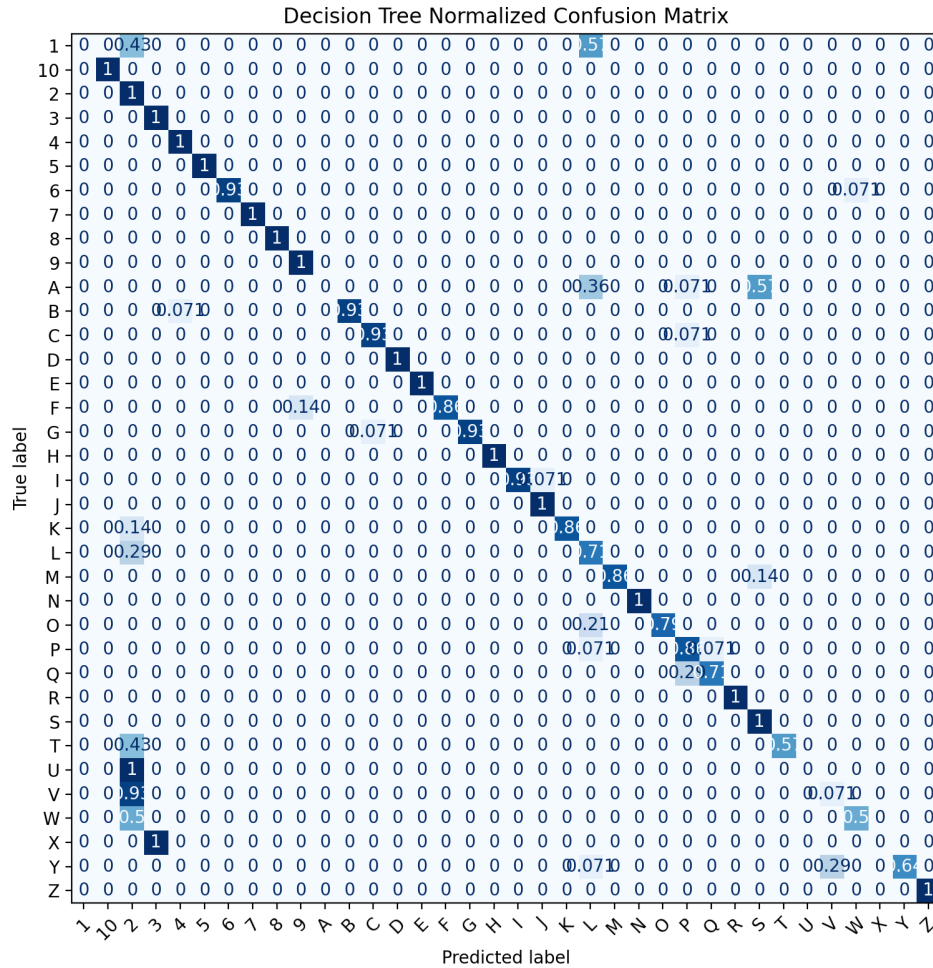


Figure 5.2.5 - Decision Tree Confusion Matrix (Normalized)

Figure 5.2.5 shows that the Decision Tree Model performed less favorably compared to the Random Forest Model with many incorrect predictions. Some letters and numbers were easily predicted (such as “e”, “z”, and the number “8”), but there was much difficulty when predicting many other letters and numbers. The letter “u” and “v”, while fairly similar, are not differentiated most likely due to positioning and slight variations in bend. Lastly the importance of each feature helped improve the model and assisted in providing higher accuracy with the Random Forest model.

The glove was then tested physically, with 10 repetitions of each gesture recorded as a pass or a fail depending on output. Overall accuracy is shown in Table 5.2.3:

Gesture	Accuracy
A	100%

B	100%
C	100%
D	100%
E	100%
F	90%
G	100%
H	100%
I	60%
J	90%
K	100%
L	100%
M	90%
N	100%
O	90%
P	90%
Q	70%
R	100%
S	100%
T	100%
U	90%
V	100%
W	90%
X	100%
Y	90%
Z	100%

1	100%
2	100%
3	100%
4	100%
5	100%
6	90%
7	100%
8	100%
9	80%
10	100%
Accuracy	97.50%

Table 5.2.3 - Gesture Statistics

Signs like P and Q are very similar as shown in appendix F ASL chart which explains their lower accuracies, 9 and F are extremely similar as well.

The battery life of the glove was tested on three separate occasions. The first was during initial testing of the device, which involved timed testing which began the moment the glove was powered on. The timing results of each battery test are shown in Table 5.2.4 - Battery Life Test Results:

Test Run #	Testing Activity Log	Result
Battery Test 1	The test began while testing the model of the glove. Glove was in use for an initial time of 38 minutes and 43 seconds. The glove was removed and then left for 2 hours and 16 minutes. The glove was then used to recognize some other gestures for another 19 minutes and 12 seconds before being left on for another 2 hours and 15 minutes before being powered off.	~5 hours 6 minutes
Battery Test 2	The test was simply to see how long the battery lasted with no usage of translation features. The glove lasted 5 hours and 6 minutes. The battery heat was measured	~5 hours 14 minutes

	every hour and was unplugged after reaching 93 degrees fahrenheit. The maximum heat a lithium battery generally reaches is 95 degrees fahrenheit (about 35 degrees celsius), so the test was ended then.	
Battery Test 3	The final test was used to see how the glove performed after a long period of time being on. The glove was left on for 3 hours before testing. The testing user made gestures for 15 minutes before stopping. Another test was done 45 minutes later and tested again for 15 minutes. The final test was done 45 minutes later and gestures were made for another 15 minutes at 5 hours of battery life.	~5 hours 4 minutes

Table 5.2.4 - Battery Life Test Results

For the response time testing we first tested the transmission latency by measuring the amount of time it took to take in the sensor data, run through the model, then send the output data to a computer. We measured the average transmission time from the output device to about 10ms and started an internal clock in the arduino code, once the process was complete a message was sent back to the ESP32 to stop the clock and output the results minus the return time. This measurement on average was less than 65ms.

5.3 - System Validation

The system performance was evaluated against the target values. Said target values stem from the system requirements found in Table 2.5 - Requirement Solution Traceability Matrix. These values are the accuracy of the models ability to predict correct gestures input by the user, the battery life of the project, safety and comfort of the user from generated battery heat, and finally the output latency of the glove and its predictions. These final validation results are shown in Table 5.3.1 - Target Requirement Evaluations:

Requirement	Target	Actual	Result
Accuracy	>90%	97.50%	Pass
Response Time	<65 ms	~52ms	Pass
Battery Life	4–6+ hours	~5 hours	Pass
User Safety	Minimal heat stress	~80°F	Pass

Table 5.3.1 - Target Requirement Evaluations

The project successfully meets each of these system requirements. The model was required to successfully and correctly predict gestures with at least 90% accuracy, which it

successfully achieves an accuracy of 97.50% as shown earlier in Table 5.2.3 - Accuracy By Gestures. The battery life was required to be four to six hours and successfully achieves that goal through the simple testing the glove was put through. The testing user reported little to no heat stress during testing, and successfully complies with the third requirement. Therefore, the project successfully meets each system requirement. The method we used for measuring latency was using the built in c function micros() we started the timer once the information was fed into the model and then outputs when the model tells us its predictions. The average of these measurements was under the 65ms requirement.

```
BLE advertising started
System ready
Predicted index: 12
Predicted label: C
Confidence: 0.359167
Top 3 predictions:
  1) C = 0.359167
  2) 5 = 0.233889
  3) D = 0.090722
```

Figure 5.3.1 - Output of Model (Arduino IDE)

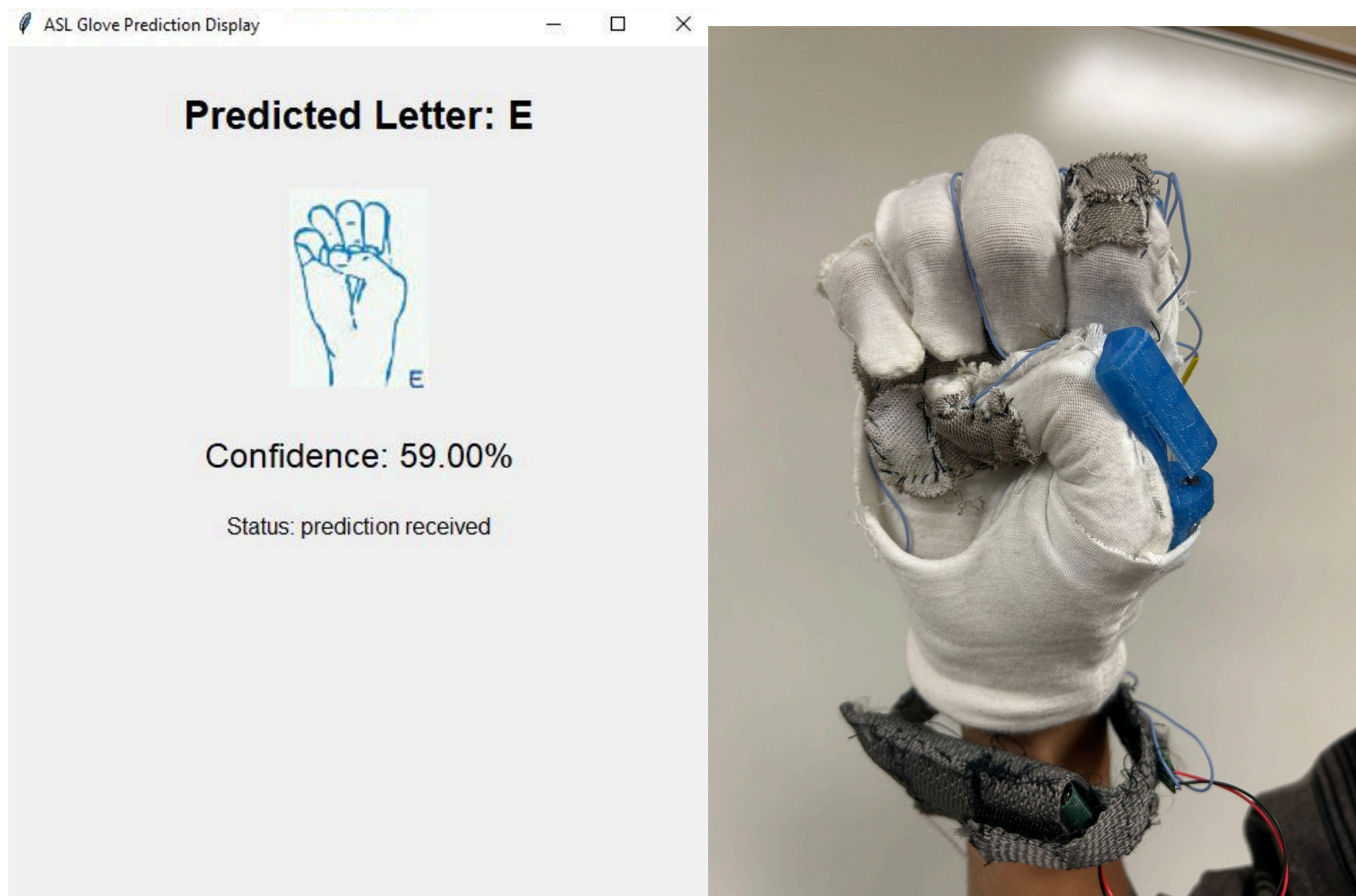


Figure 5.3.2 - Output of Model (BLE Output)

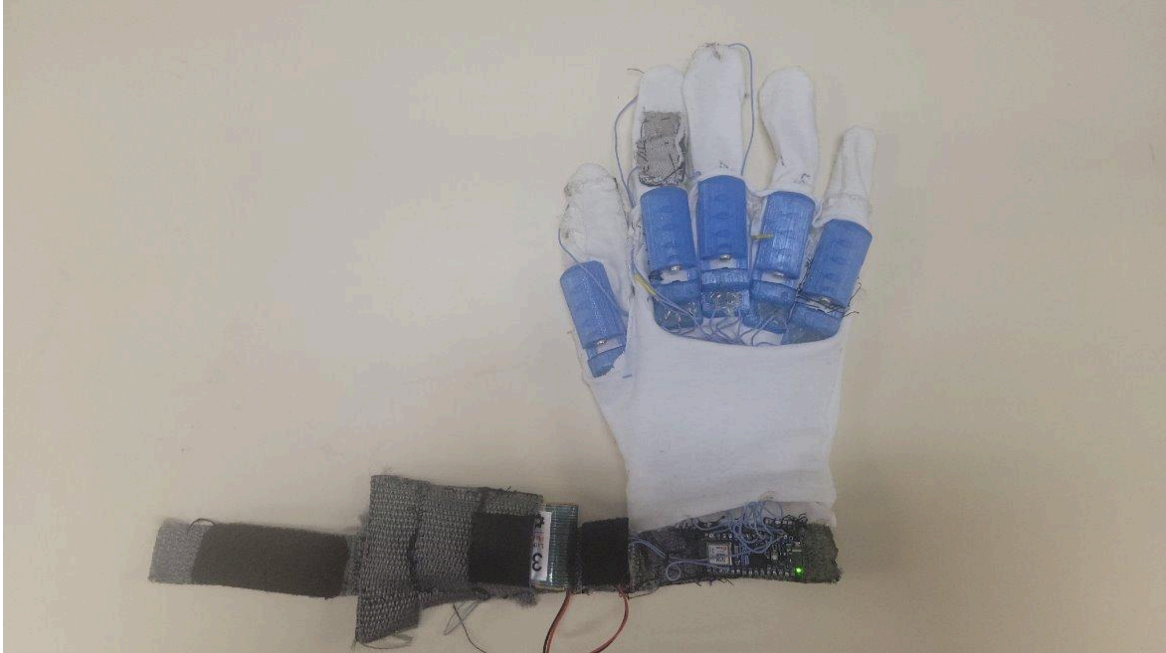


Figure 5.3.3 - Final Prototype Design

Chapter 6 – Broader Impacts of the Project

The wearable glove for ASL gesture recognition has broad impacts that can affect any community that uses a gesture-based communication system. Language is an important aspect of any social group and integrity of communication needs to be protected. This tenet guided the development of the glove.

The main ethical consideration of the development of the glove was the social impact of the technology on the ASL community. Extreme care and caution needed to be maintained in order to ensure the accuracy of the gesture recognition. Publicly available ASL diagrams showcase correct gestures and results were checked thoroughly against publicly available sources. Due to the possibility of creating easily available transcription systems for gesture-based languages, we expect positive cultural and social impacts.

The safety of the user is also of utmost concern. The only part of the design with a non-negligible safety risk would be the battery, which is a concern with any wearable electronic device. Extra fabric was implemented in order to prevent heat transfer and no direct contact should be allowed. If a commercial unit were to be created, a safe charging circuit and battery chassis is necessary. The unit should pose no health risks when properly used.

Social and Safety factors were of the utmost importance and were addressed according to their weights as seen in the Top-Level Weighted Decision Matrix (Table 3.1.2). Other impacts, namely those of economic and environmental natures, are expected to be minimal. With a refined approach units could cost around \$150 to produce. The glove also uses minimal electronic components and can be expected to have a very small environmental impact if disposed of properly.

Overall, the wearable glove for ASL gesture recognition is expected to have either at least some impact or a strong positive impact in all considered frameworks.

Chapter 7 – Conclusions

7.1 - Conclusion

The ASL glove demonstrates a real-time, portable system capable of translating hand gestures into readable outputs. It implements a model deployed onto an Arduino Nano ESP32 and uses I2C to communicate with the three main subsystem sensor systems. The system achieves an accuracy of 97.50% and maintains steady battery life and supports safety and health of the user wearing the glove. Therefore, all system requirements are successfully met and the system functions appropriately.

7.2 - Future Work

Full word gesture recognition is the next major step of this project. The IMU provides several of the most important features (via Random Forest) and would be the most important subsystem when recognizing full word gestures. Most ASL gestures and words require large movements involving the arms and wrists, which require the 9-Axes provided by the IMU using the euler and accelerometer measurements. The gyroscope provides much needed angular measurements as well. The glove would rely heavily on the IMU and spatial awareness to the body and words that require the other hand. A separate glove similar in architecture to the system that we've created would go on the other hand with each subsystem on the main glove on the left hand. Placement of the microcontroller would change and would require a centralized unit that could establish a connection between the arms and on the torso to provide full movement of each arm while wired up to each sensor. Along with another system on the left hand, an ASL user would have to interact with the production of the system in order to ensure accurate translation, and ethical consideration of ASL users. An alternative microcontroller would be preferred for the scope of this future work.

Dynamic gesture recognition would require intense IMU usage as well as adept real-time reading of sensory data in which the microcontroller can scan a constant number of previous lines and make predictions using that input. An alternative microcontroller would be required to handle the large amount of IMU data. Dynamic gesture recognition can be attained with the current system implemented, but this would require further testing and training that went beyond the scope of the project.

Multi-language support would require separate models to be trained. This is also feasible, but would require further training and testing beyond the scope of the project. This would require international sign language users in order to ensure proper representation of other international sign language users.

7.3 - Challenges

Several challenges were faced during the testing of the prototype. Some hardware issues arose during the initial assembly of the glove. Some connections would break causing sensory issues. For instance, the ground connection to the ADC with its address tied LOW (the ADC used for the hall effect sensor for the thumb) would disconnect. When disconnecting, the sensory data would appear to be normal until the thumb was completely bent. When bending fully, the data sent through serial would read maximum bend values, but every other line would have the digital value fluctuate between the max bend value and zero.

During testing, several of the models that were trained would incorrectly predict values and decrease the overall accuracy. Some of these common missed predictions were 'F', 'I', 'L', 'M', 'N', 'P', 'Q', 'S', 'U', 'Y', and '9'. The gestures necessary for letters 'F' and '9' are extremely similar. During acquisition of data for the developing the training/testing set a minor difference was made for distinguishing the two gestures. For 'F' the thumb and index are rounded with the fingers spread out similar to an 'OK' gesture. By contrast, '9' has the thumb and index pursed making a similar motion to a pinch. However, this did not seem to provide much of a difference. The model does not distinguish based on the position of the thumb and index finger due to similar hall effect sensor values. However, for some of the 'F' sensory data, the contact pad on the index finger and thumb did not connect, therefore the model is more likely to identify and correctly predict 'F' when there is no contact on the index finger pad. The result is an inconsistent prediction of 'F' while '9' is correctly predicted.

The hall effect sensor located on the pinky became loose against the glove when testing. This led to inconsistent predictions on gestures such as 'A', 'I', 'S', and 'Y' due to the similar shape of each. While 'A' and 'S' were correctly and accurately predicted, 'I' and 'Y' weren't predicted successfully due to the hall effect sensor located on the pinky. It would not bend properly when straightening out and would in fact bend as if the pinky was being bent. Sowing down the hall effect sensor the glove provided better results, but the hall effect sensor still doesn't provide completely accurate input, causing some variation when predicting.

7.4 - Final Remarks

This project demonstrates the feasibility of combining embedded systems and machine learning to create assistive communication technology. The project could be improved to provide durable equipment that aids in the communication of ASL users by recognizing full words and gestures, thus allowing for deeper, meaningful communication.

Chapter 8 – References

[1] K. K. Kim *et al.*, “A simplified wearable device powered by a generative EMG network for hand-gesture recognition and gait prediction,” *Nature Sensors*, 2025.

[2] E. Ramsden, *Hall-Effect Sensors: Theory and Application*, 2nd ed. Amsterdam, Netherlands: Elsevier, 2006.

[3] Signing Savvy, LLC, “Sign Word List for NUMBERS,” Signing Savvy, 2019.
<https://www.signingsavvy.com/wordlist/numbers>

[4] American Society for Deaf Children, “Free ASL Alphabet Chart,” Jun. 10, 2019. [Online]. Available: <https://deafchildren.org/2019/06/free-asl-alphabet-chart/>.

Appendices

Appendix A - Work Breakdown Structure (WBS)

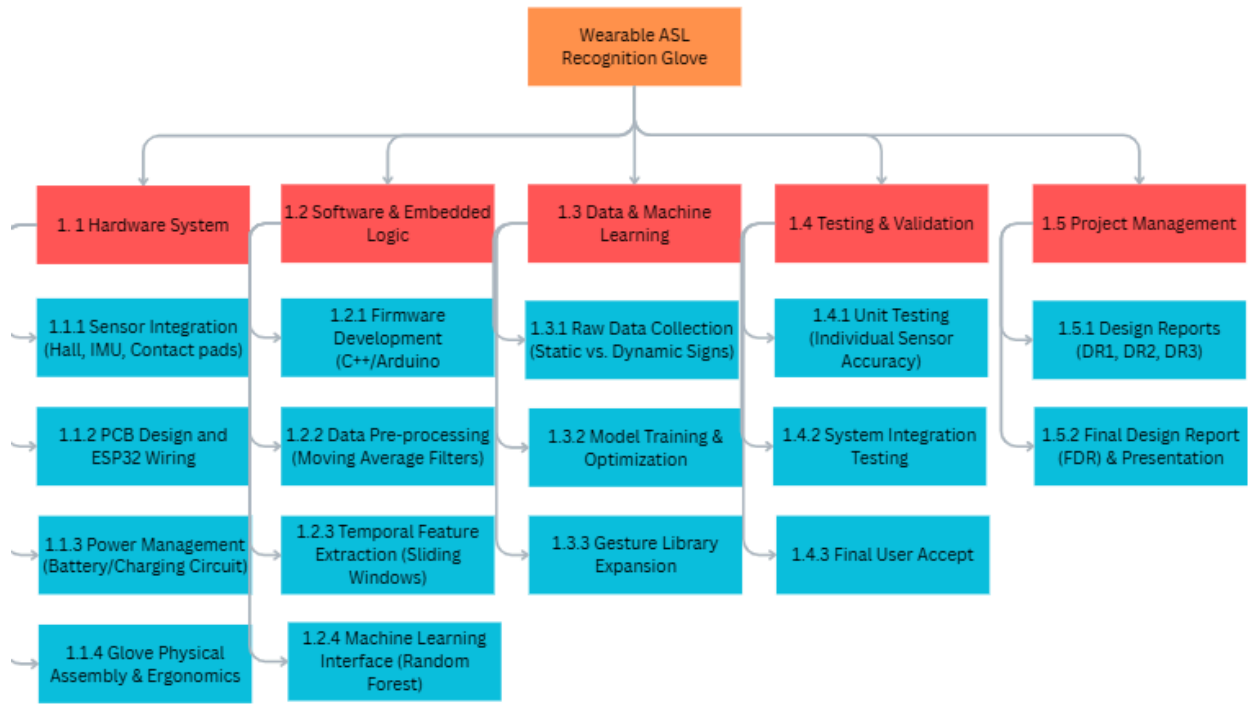


Figure A.1 - Work Breakdown Structure

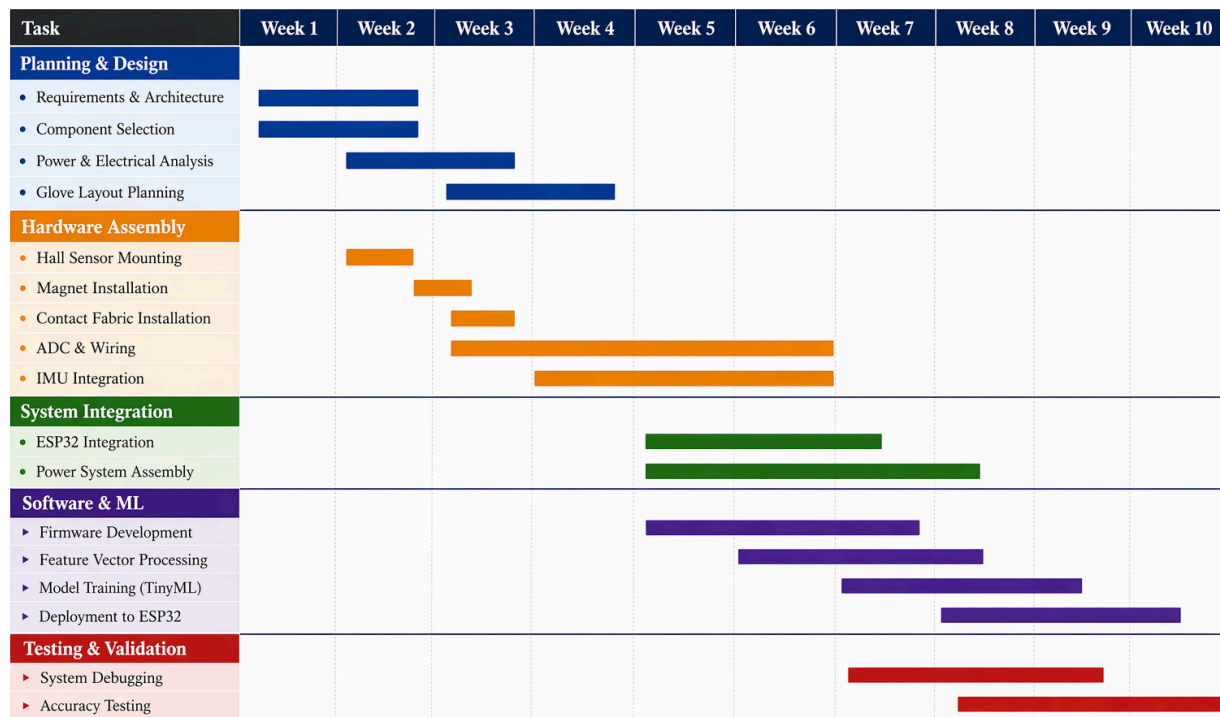


Figure A.2 - Work Breakdown Gantt Chart

Appendix B - User Manual

1. Introduction

The Wearable Glove for American Sign Language (ASL) Gesture Recognition system is a portable, battery-powered assistive prototype designed to recognize and translate the ASL alphabet and the numbers 1 through 10 in real time. The system uses an Arduino Nano ESP32, five Hall effect sensors, contact fabric sensors, and an Adafruit BNO055 IMU to collect hand and wrist data. This data is processed by an onboard lightweight Random Forest / TinyML model, and the resulting gesture prediction is sent to an external display device through BLE communication.

The glove was designed to act as a teaching and translation aid. It distinguishes both static gestures such as A, R, and 1, and dynamic gestures such as J, Z, and 10. The final system achieved an overall gesture accuracy of 97.50%, a response time of less than 65 ms, and a battery life of about 4 to 6 hours, satisfying the project requirements.

2. System Overview

The glove system consists of the following major subsystems:

2.1 Microcontroller Subsystem

The central controller is the Arduino Nano ESP32, which receives all sensor data, runs the trained model, and transmits the output through BLE.

2.2 Hall Effect Sensor Subsystem

Five Hall effect sensors are used to measure finger bend. One sensor is placed across each metacarpal joint, and magnets attached along the fingers allow bend to be estimated from magnetic field changes.

2.3 Contact Sensor Subsystem

Conductive fabric contact pads are used to detect special finger-to-finger and thumb-to-palm contacts. These help distinguish gestures that would otherwise appear similar, such as R, P/Q, and some number gestures.

2.4 IMU Subsystem

The Adafruit BNO055 Absolute Orientation IMU tracks wrist position and motion. This is important for distinguishing gestures with similar finger shapes but different hand orientation or motion.

2.5 Power Subsystem

The glove is powered by a 3.7 V Li-Po battery regulated down to 3.3 V using a buck converter. The design is intended for portable use.

2.6 BLE Output Subsystem

After classification, the prediction and confidence are transmitted to a computer or display device over Bluetooth Low Energy (BLE). After model deployment, only prediction-related output is streamed from the glove.

3. Materials Included in the Prototype

The user should verify that the following major components are present before operation:

- Arduino Nano ESP32
- Adafruit BNO055 IMU
- Two ADS1115 I2C ADC modules
- Five APS11760LUAA linear Hall effect sensors
- Five neodymium magnets
- Conductive fabric / thread contact pads
- 3.7 V Li-Po battery
- 3.3 V buck regulator
- Cotton glove base
- Sensor wiring and insulation materials

4. Safety Information

Before operating the glove, the following precautions should be observed:

1. Do not allow the battery or exposed wiring to directly contact the skin. Insulation and layered glove material are important for user safety.
2. Do not operate the glove if wires are loose, frayed, or disconnected.
3. Do not bend the glove sharply at wired sections, since this may damage sensor connections.
4. Disconnect power immediately if excessive heat is felt.
5. Use only the intended voltage levels. The ESP32 subsystem is designed around a 3.3 V supply path.

5. Setup Instructions

5.1 Wearing the Glove

1. Place the glove on the correct hand used during prototype testing.

2. Ensure each finger is fully inserted into its proper position.
3. Confirm that the Hall effect sensors align with the first knuckle region of each finger.
4. Confirm that the contact pads rest at the intended locations:
 - thumb contact areas
 - middle and forefinger contact area
 - palm contact area
5. Make sure the IMU rests securely near the wrist region.

5.2 Powering the Device

1. Verify the battery is charged.
2. Confirm the buck converter is connected properly between the battery and ESP32.
3. Turn on the glove power.
4. Wait a few seconds for the ESP32 and sensor subsystems to initialize.

5.3 Connecting to the Display Device

1. Open the BLE receiver or companion display program on the external computer or device.
2. Pair the external device with the ESP32 glove.
3. Confirm that the BLE connection is active before attempting translation.
4. When connected properly, the device should begin receiving model predictions from the glove.

6. Operating Instructions

6.1 Starting Translation

After the glove is powered and connected:

1. Place the hand in a neutral resting position.
2. Form a supported ASL letter or number gesture.
3. Hold the gesture still briefly so the sensors can stabilize.
4. For dynamic gestures such as J, Z, or 10, complete the required wrist or hand motion naturally.
5. Wait for the output to appear on the connected display device.

6.2 How the Glove Interprets Gestures

The model uses a 18-feature input vector consisting of:

- 5 Hall effect sensor features
- 5 contact sensor features
- 9 IMU-related features

These features are processed by the onboard lightweight model to generate a gesture prediction. The glove is intended to classify:

- A through Z
- 1 through 10

6.3 Best Practices for Accurate Use

For best performance:

- Keep the glove snug on the hand.
- Make deliberate, clearly formed gestures.
- Avoid partially formed finger positions.
- Ensure the thumb contacts the intended pad when required.
- For orientation-dependent gestures like P, Q, V/2, or C/O, keep wrist position consistent and intentional.

7. Performance Notes

During testing, the system achieved the following overall results:

- Accuracy: 97.50%
- Response Time: less than 65 ms
- Battery Life: approximately 4 to 6 hours
- User Safety: minimal heat stress reported during operation

Some gestures were recognized more reliably than others. Gestures such as A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, R, W, X, Y, 1, 3, 4, 5, 6, 7, 8, 9, and 10 performed strongly, while P, Q, S, T, V, and 2 showed lower accuracy during physical testing. The human hand, as usual, refused to behave like a perfectly cooperative engineering component.

8. Maintenance and Care

To preserve operation of the prototype:

1. Disconnect power when the glove is not in use.
2. Do not wash the glove while electronic components are attached.
3. Store the glove in a dry location.
4. Avoid crushing the Hall sensors or dislodging the magnets.
5. Periodically inspect:
 - wiring
 - battery condition
 - contact pad stitching
 - sensor placement

6. Re-secure any loose conductive fabric before reuse.

9. Troubleshooting

9.1 No Output Appears on Display

Possible causes:

- BLE connection not established
- glove power is off
- battery voltage too low
- ESP32 code not running correctly

Corrective actions:

- verify the battery is connected and charged
- reconnect the BLE receiver device
- restart the glove power
- re-upload firmware if necessary

9.2 Incorrect Gesture is Predicted

Possible causes:

- glove is misaligned on the hand
- contact pads are not touching correctly
- magnets or Hall sensors have shifted
- wrist orientation differs from training conditions

Corrective actions:

- reposition the glove
- reform the gesture more clearly
- inspect sensor alignment
- repeat the gesture with steadier wrist position

9.3 Dynamic Gestures Are Not Recognized Properly

Possible causes:

- motion was too small or too fast
- IMU readings were inconsistent
- gesture was not completed fully

Corrective actions:

- repeat the full wrist motion
- slow the gesture slightly
- keep the glove firmly seated at the wrist

9.4 Device Becomes Warm

Possible causes:

- prolonged operation
- battery nearing discharge
- insufficient insulation

Corrective actions:

- power off the glove
- allow the unit to cool
- inspect insulation between hardware and skin
- discontinue use if abnormal heating continues

10. Limitations

This prototype is intended as a proof-of-concept educational and assistive system. It is not yet a complete commercial translation product. Current limitations include:

- support is limited to letters A-Z and numbers 1-10
- some gestures are more difficult to distinguish than others
- battery life is limited to several hours
- performance depends on proper fit, alignment, and calibration
- full-word or two-handed ASL recognition is not yet implemented

11. Expected Future Improvements

Future work may include:

- full-word gesture recognition
- greater dependence on spatial movement tracking
- support for both hands
- centralized hardware layout for larger body motion tracking
- further refinement with direct ASL user involvement

12. Summary of User Procedure

For regular use, the operator should follow this sequence:

1. Inspect the glove for safe wiring and secure sensors.
2. Put on the glove and align all sensors correctly.

3. Power the glove using the battery subsystem.
4. Connect the glove to the external BLE display device.
5. Form a supported ASL gesture.
6. Wait for the model prediction to appear.
7. Repeat for additional gestures.
8. Power off the glove after use.

Appendix C - Knowledge Skill Table

Course	Knowledge/Skills
COSC 1336	Basics of C programming
CMPE 3301	Introduction to machine learning and programming with ESP32 and Arduino IDE
CMPE 4342	Learning about network communication and communication protocols through bluetooth
EENG 4307	Learning about microprocessors and their basic structure
EENG 3304	Learned how to solder electronics together
CPME 4333	Learned how to properly build a machine learning model and what each metric means

Table C.1 - Knowledge Skill Table

Appendix D - Hardware and Features

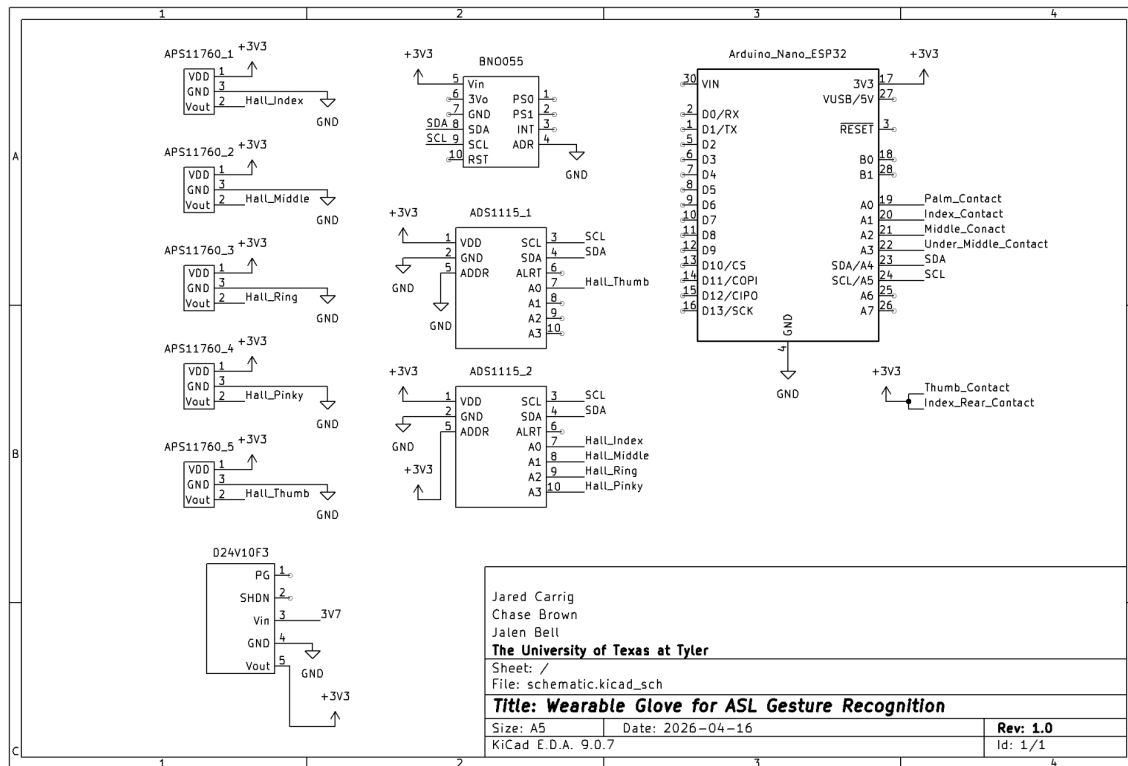


Figure D.1 - Full Design Schematic

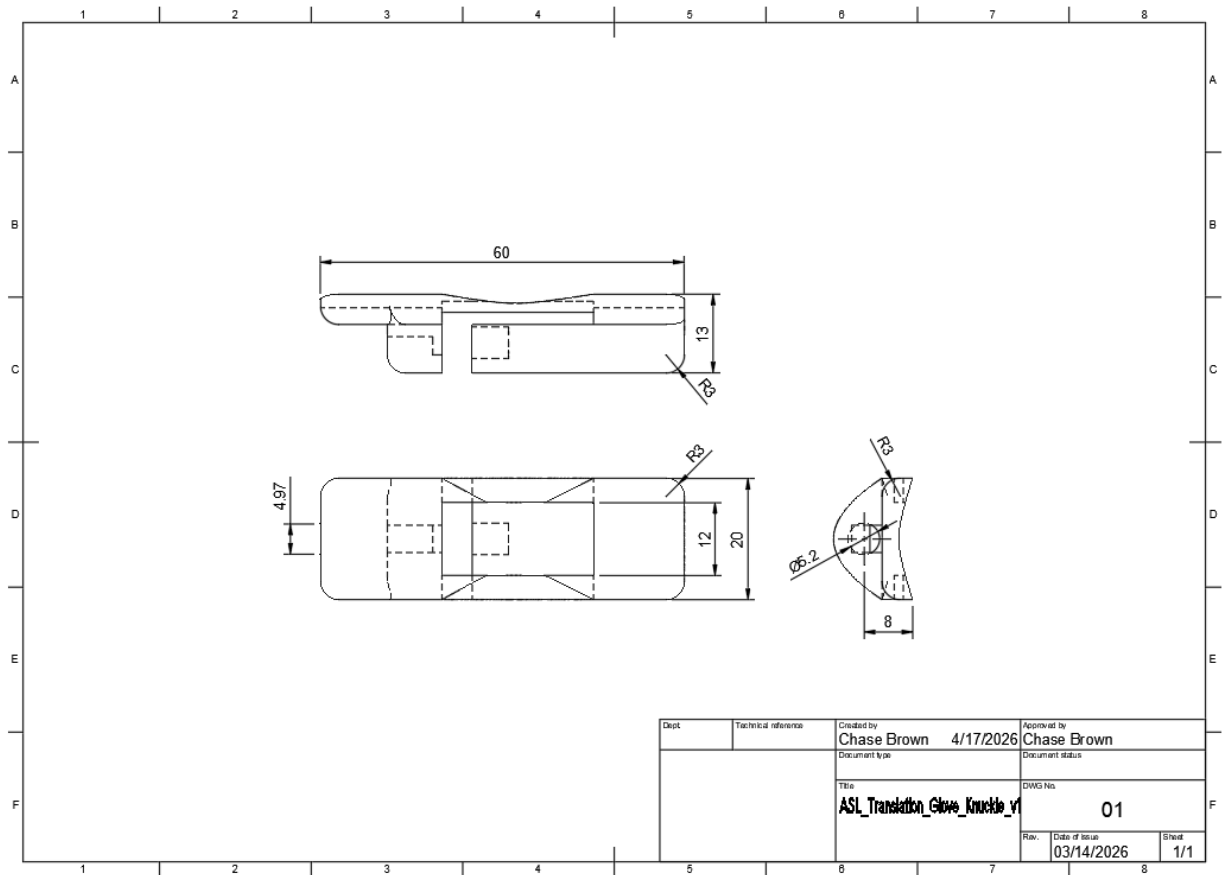


Figure D.2 - Hall Effect Sensor Finger Housing Schematic

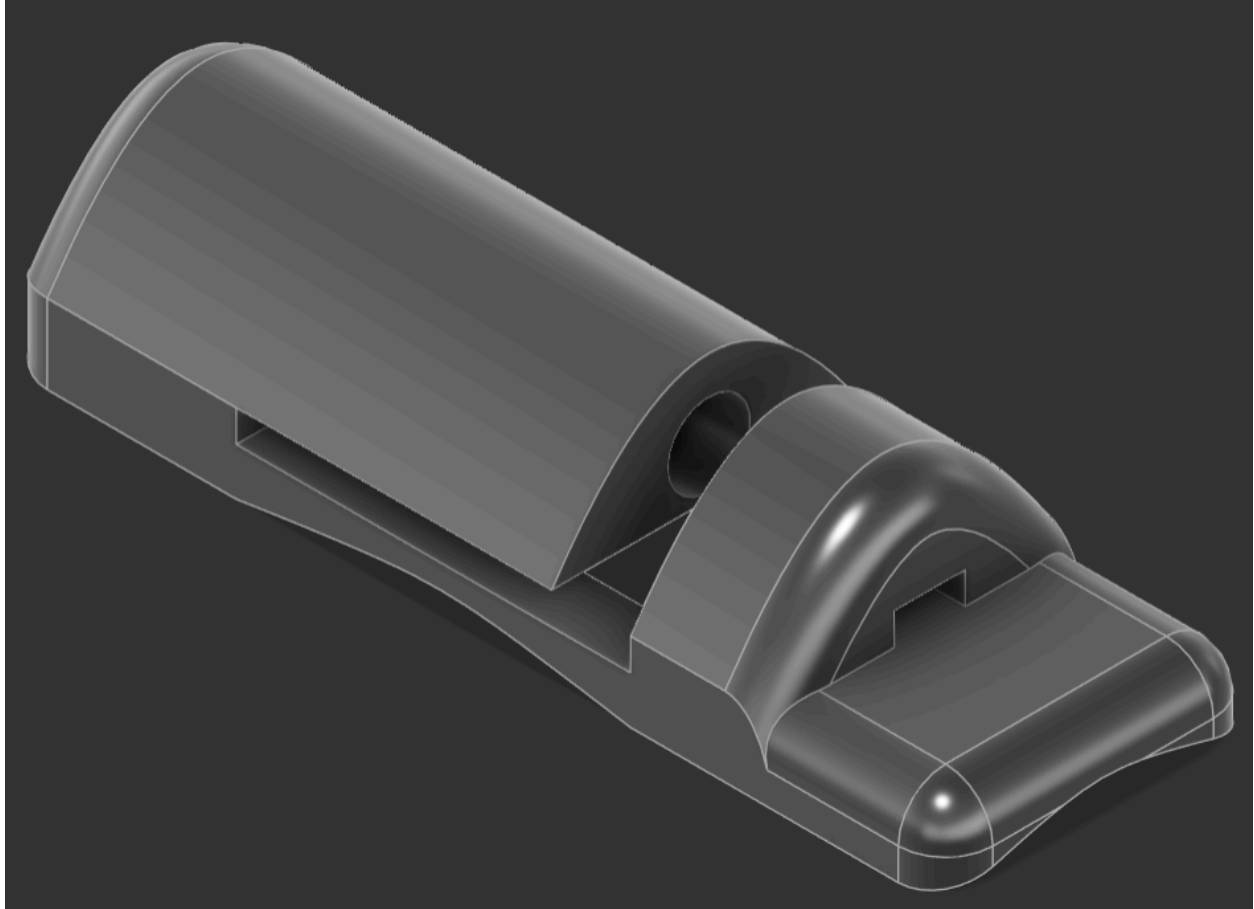


Figure D.3 - Hall Effect Sensor Finger Housing

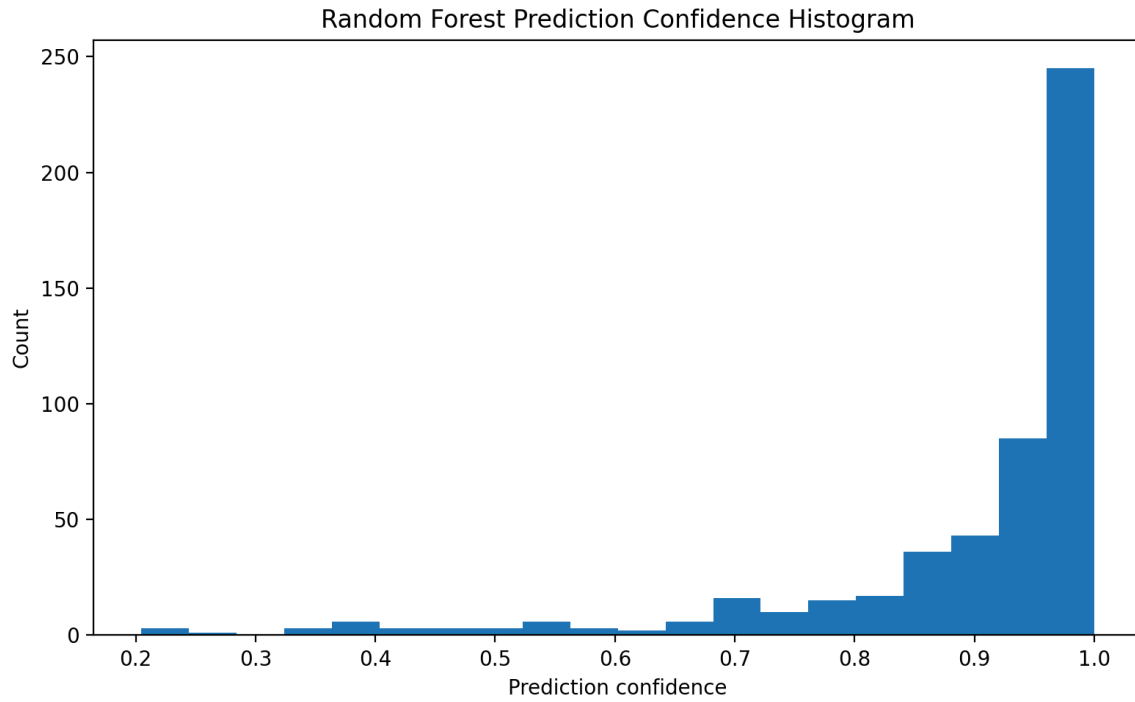


Figure D.4 - Random Forest Prediction Confidence Histogram

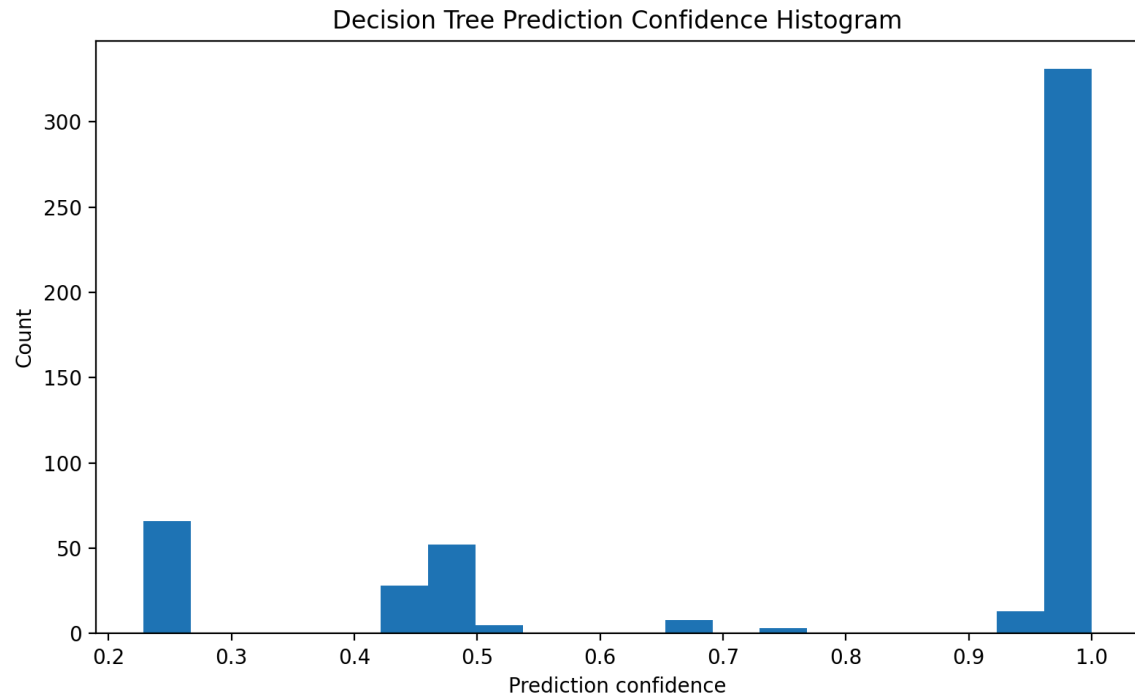


Figure D.5 - Decision Tree Prediction Confidence Histogram

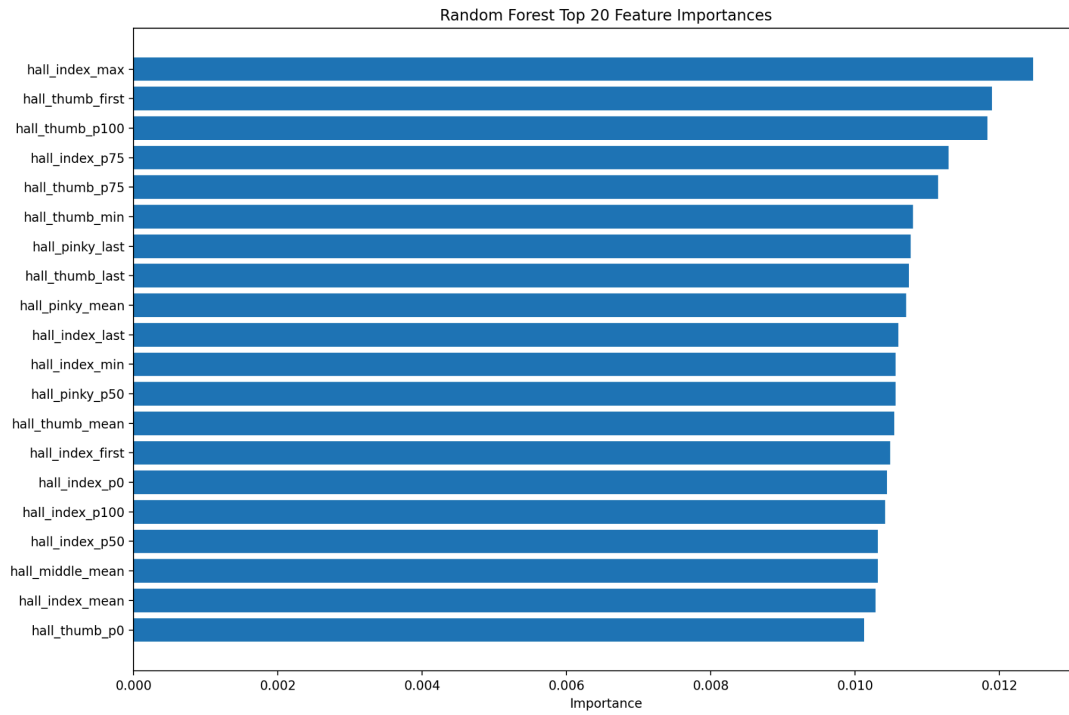


Figure D.6 - Random Forest Feature Importances

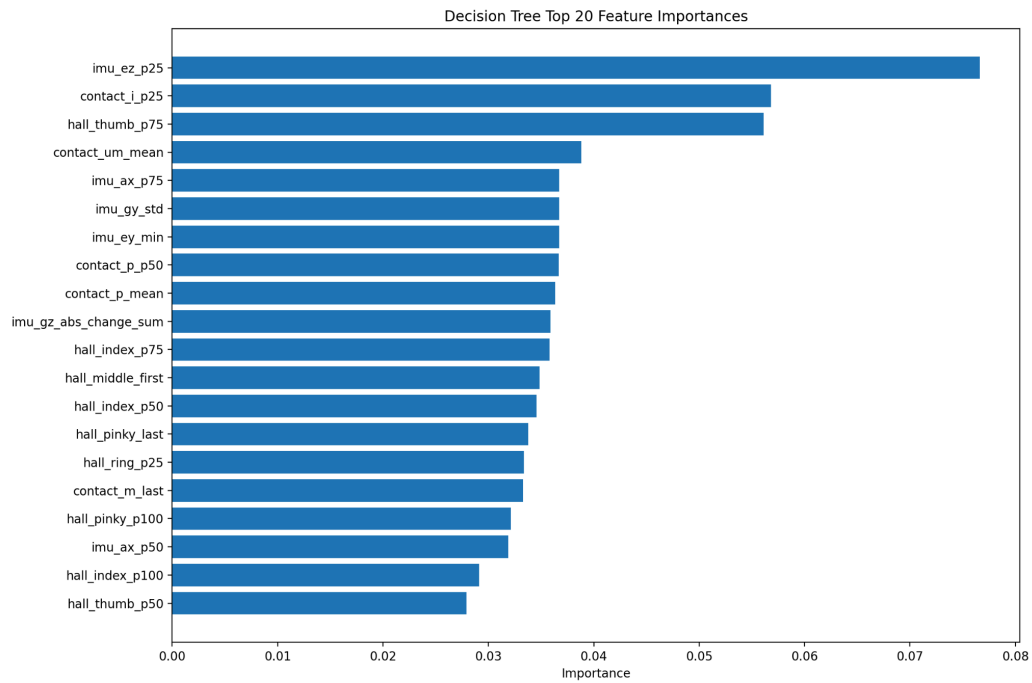


Figure D.7 - Decision Tree Feature Importances

Per-sensor fabricated features

For every raw sensor signal like `hall_thumb`, `imu_ax`, `contact_p`, etc., the code creates these:

- `_first`: the first recorded value in the sequence
- `_last`: the last recorded value in the sequence
- `_mean`: the average value over the whole sequence
- `_min`: the smallest value in the sequence
- `_max`: the largest value in the sequence
- `_std`: how much the values vary from the average
- `_range`: the difference between the max and min values
- `_delta`: change from first value to last value
- `_abs_change_sum`: total amount of movement/change across the sequence, ignoring direction
- `_slope`: rate of change from first value to last value over time
- `_p0`: value at the start of the sequence
- `_p25`: approximate value at 25% of the way through the sequence
- `_p50`: approximate middle value of the sequence
- `_p75`: approximate value at 75% of the way through the sequence
- `_p100`: value at the end of the sequence

Whole-sequence fabricated features

These are added once per gesture sequence:

- `duration_s`: total time the gesture lasted in seconds
- `num_frames`: number of recorded frames in the sequence

Acceleration magnitude

Built from `imu_ax`, `imu_ay`, `imu_az`

- `accel_mag_mean`: average total acceleration magnitude
- `accel_mag_max`: highest acceleration magnitude
- `accel_mag_std`: variation in acceleration magnitude
- `accel_mag_abs_change_sum`: total change in acceleration magnitude over time

Gyroscope magnitude

Built from `imu_gx`, `imu_gy`, `imu_gz`

gyro_mag_mean: average total rotational motion
gyro_mag_max: highest rotational motion
gyro_mag_std: variation in rotational motion
gyro_mag_abs_change_sum: total change in rotational motion over time
Magnetometer magnitude

Built from imu_mx, imu_my, imu_mz

mag_mag_mean: average magnetic field magnitude
mag_mag_max: highest magnetic field magnitude
mag_mag_std: variation in magnetic field magnitude
mag_mag_abs_change_sum: total change in magnetic field magnitude over time

Figure D.8 - Feature Explanation

Appendix E - Code

The link to the github repository where you can find all the code used in this project is:

<https://github.com/CBrownUTT/American-Sign-Language-Translation-Glove>

The QR Code:



```
=====
```

ESP32 5x A1324 Hall Effect Sensors Test EXAMPLE ASL Translating Glove

This code:

1. Initializes 5 A1324 Hall Effect Sensors
2. Reads the raw data from the sensors
3. Translated the raw data into readable variables
4. Formatted and sent the read data through serial to the attached device

The following code snippet was implemented for use in the testing of A1324 Hall Effect Sensors. Proper testing, calibration and evaluation is vital to the accuracy of the product. After using this code to properly assess each sensor for proper accuracy and function. By using this code, the project could continue on to the next steps of the final goal of an American Sign Language translating glove. Hall effect sensors are vital in determining how much each finger is bent whether the finger is halfway bent or not. This code lays the groundwork for future revisions of the code.

Platform: ESP32 (Arduino Framework)

```
=====
*/
```

```
#define NUM_SENSORS 5
```

```
// Change pins if needed
```

```
int hallPins[NUM_SENSORS] = {A0, A1, A2, A3, A4};
```

```
const float SENSITIVITY = 5.0; // mV/G for A1324
```

```
float zeroVoltage[NUM_SENSORS];
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    delay(1000); //delays for 1 second
```

```
    Serial.println("5x A1324 Hall Sensors - Calibrated Mode");
```

```
    Serial.println("Remove magnets. Calibrating...");
```

```

delay(2000);

// Calibrate zero field for each sensor
for (int s = 0; s < NUM_SENSORS; s++) {
    long sum = 0;

    for (int i = 0; i < 200; i++) {
        sum += analogReadMilliVolts(hallPins[s]);
        delay(2);
    }

    zeroVoltage[s] = (sum / 200.0) / 1000.0;

    Serial.print("Sensor ");
    Serial.print(s + 1);
    Serial.print(" Zero Voltage: ");
    Serial.print(zeroVoltage[s], 4);
    Serial.println(" V");
}

Serial.println("-----");
}

void loop() {

    for (int s = 0; s < NUM_SENSORS; s++) {

        uint32_t milliVolts = analogReadMilliVolts(hallPins[s]);
        float voltage = milliVolts / 1000.0;

        float magneticField = (voltage - zeroVoltage[s]) / (SENSITIVITY /
1000.0);

        Serial.print("S");
        Serial.print(s + 1);
        Serial.print(": ");

        Serial.print(voltage, 3);
        Serial.print(" V | ");
    }
}

```

```

    Serial.print(magneticField, 1);
    Serial.print(" G    ");
}

Serial.println();
Serial.println("-----");

delay(250);
}

=====
ESP32 Model Calling Function
ASL Translating Glove

This code:
1. Reads data from contact sensors, hall effect sensors and an
Inertial Motion Unit (IMU)
2. Inserts the data into the start of the random forest model
3. The random forest model processes the data
4. Then the final decision is output at the end of the code

Platform: ESP32 (Arduino Framework)
=====

This function implements a random forest model.
It returns a detected ASL letter based on:
- Contact sensors (first priority)
- Finger bend levels
- Palm orientation
#include <Wire.h> // Lets the board use I2C
#include <Adafruit_ADS1X15.h> // Library for ADS1115 ADC boards
#include <Adafruit_BNO055.h> // Library for the BNO055 IMU
#include <Adafruit_Sensor.h> // Helper sensor library
#include <math.h> // Needed for sqrt
#include <string.h> // Needed for memset
#include "rf_model.h" // Your exported random forest model

// ----- Contact Pins -----

```



```

#define PALM          17 // Palm contact sensor pin
#define INDEX         18 // Index contact sensor pin
#define MIDDLE        19 // Middle contact sensor pin
#define UNDER_MIDDLE 20 // Under-middle contact sensor pin

// ----- Timing -----
#define SAMPLE_DELAY_MS 20 // 20 ms between samples gives about 50 Hz
#define MAX_FRAMES 25 // Number of frames to store for one gesture
window

// ----- Devices -----
Adafruit_ADS1115 ads1; // First ADS1115 at 0x48
Adafruit_ADS1115 ads2; // Second ADS1115 at 0x49
Adafruit_BNO055 bno28(55, 0x28); // BNO055 option at address 0x28
Adafruit_BNO055 bno29(56, 0x29); // BNO055 option at address 0x29
Adafruit_BNO055* bno = nullptr; // Pointer to whichever BNO055 is
found

// ----- Raw Frame Struct -----
// This holds one raw sensor sample from the glove
struct RawFrame {
    unsigned long timestamp_ms; // Time this sample was taken

    float hall_thumb; // Thumb hall sensor
    float hall_index; // Index hall sensor
    float hall_middle; // Middle hall sensor
    float hall_ring; // Ring hall sensor
    float hall_pinky; // Pinky hall sensor

    float imu_ax; // Accelerometer X
    float imu_ay; // Accelerometer Y
    float imu_az; // Accelerometer Z

    float imu_gx; // Gyroscope X
    float imu_gy; // Gyroscope Y
    float imu_gz; // Gyroscope Z

    float imu_mx; // Magnetometer X
    float imu_my; // Magnetometer Y

```

```

float imu_mz; // Magnetometer Z

float contact_p; // Palm contact
float contact_i; // Index contact
float contact_m; // Middle contact
float contact_um; // Under-middle contact
};

// ----- Frame Buffer -----
// This stores one full gesture sequence before feature extraction
RawFrame g_frames[MAX_FRAMES]; // Array of raw samples
int g_frameCount = 0; // Current number of stored frames

// ----- Feature Map Enum -----
// These indexes define exactly where each feature goes in the
// 284-element model input
// The order must match RFModel::FEATURE_NAMES exactly
enum FeatureIndex {
    IDX_duration_s = 0,
    IDX_num_frames = 1,

    IDX_hall_thumb_first = 2,
    IDX_hall_thumb_last,
    IDX_hall_thumb_mean,
    IDX_hall_thumb_min,
    IDX_hall_thumb_max,
    IDX_hall_thumb_std,
    IDX_hall_thumb_range,
    IDX_hall_thumb_delta,
    IDX_hall_thumb_abs_change_sum,
    IDX_hall_thumb_slope,
    IDX_hall_thumb_p0,
    IDX_hall_thumb_p25,
    IDX_hall_thumb_p50,
    IDX_hall_thumb_p75,
    IDX_hall_thumb_p100,

    IDX_hall_index_first,
    IDX_hall_index_last,

```

```
IDX_hall_index_mean,  
IDX_hall_index_min,  
IDX_hall_index_max,  
IDX_hall_index_std,  
IDX_hall_index_range,  
IDX_hall_index_delta,  
IDX_hall_index_abs_change_sum,  
IDX_hall_index_slope,  
IDX_hall_index_p0,  
IDX_hall_index_p25,  
IDX_hall_index_p50,  
IDX_hall_index_p75,  
IDX_hall_index_p100,  
  
IDX_hall_middle_first,  
IDX_hall_middle_last,  
IDX_hall_middle_mean,  
IDX_hall_middle_min,  
IDX_hall_middle_max,  
IDX_hall_middle_std,  
IDX_hall_middle_range,  
IDX_hall_middle_delta,  
IDX_hall_middle_abs_change_sum,  
IDX_hall_middle_slope,  
IDX_hall_middle_p0,  
IDX_hall_middle_p25,  
IDX_hall_middle_p50,  
IDX_hall_middle_p75,  
IDX_hall_middle_p100,  
  
IDX_hall_ring_first,  
IDX_hall_ring_last,  
IDX_hall_ring_mean,  
IDX_hall_ring_min,  
IDX_hall_ring_max,  
IDX_hall_ring_std,  
IDX_hall_ring_range,  
IDX_hall_ring_delta,  
IDX_hall_ring_abs_change_sum,
```

```
IDX_hall_ring_slope,  
IDX_hall_ring_p0,  
IDX_hall_ring_p25,  
IDX_hall_ring_p50,  
IDX_hall_ring_p75,  
IDX_hall_ring_p100,  
  
IDX_hall_pinky_first,  
IDX_hall_pinky_last,  
IDX_hall_pinky_mean,  
IDX_hall_pinky_min,  
IDX_hall_pinky_max,  
IDX_hall_pinky_std,  
IDX_hall_pinky_range,  
IDX_hall_pinky_delta,  
IDX_hall_pinky_abs_change_sum,  
IDX_hall_pinky_slope,  
IDX_hall_pinky_p0,  
IDX_hall_pinky_p25,  
IDX_hall_pinky_p50,  
IDX_hall_pinky_p75,  
IDX_hall_pinky_p100,  
  
IDX_imu_ax_first,  
IDX_imu_ax_last,  
IDX_imu_ax_mean,  
IDX_imu_ax_min,  
IDX_imu_ax_max,  
IDX_imu_ax_std,  
IDX_imu_ax_range,  
IDX_imu_ax_delta,  
IDX_imu_ax_abs_change_sum,  
IDX_imu_ax_slope,  
IDX_imu_ax_p0,  
IDX_imu_ax_p25,  
IDX_imu_ax_p50,  
IDX_imu_ax_p75,  
IDX_imu_ax_p100,
```

```
IDX_imu_ay_first,  
IDX_imu_ay_last,  
IDX_imu_ay_mean,  
IDX_imu_ay_min,  
IDX_imu_ay_max,  
IDX_imu_ay_std,  
IDX_imu_ay_range,  
IDX_imu_ay_delta,  
IDX_imu_ay_abs_change_sum,  
IDX_imu_ay_slope,  
IDX_imu_ay_p0,  
IDX_imu_ay_p25,  
IDX_imu_ay_p50,  
IDX_imu_ay_p75,  
IDX_imu_ay_p100,
```

```
IDX_imu_az_first,  
IDX_imu_az_last,  
IDX_imu_az_mean,  
IDX_imu_az_min,  
IDX_imu_az_max,  
IDX_imu_az_std,  
IDX_imu_az_range,  
IDX_imu_az_delta,  
IDX_imu_az_abs_change_sum,  
IDX_imu_az_slope,  
IDX_imu_az_p0,  
IDX_imu_az_p25,  
IDX_imu_az_p50,  
IDX_imu_az_p75,  
IDX_imu_az_p100,
```

```
IDX_imu_gx_first,  
IDX_imu_gx_last,  
IDX_imu_gx_mean,  
IDX_imu_gx_min,  
IDX_imu_gx_max,  
IDX_imu_gx_std,  
IDX_imu_gx_range,
```

```
IDX_imu_gx_delta,  
IDX_imu_gx_abs_change_sum,  
IDX_imu_gx_slope,  
IDX_imu_gx_p0,  
IDX_imu_gx_p25,  
IDX_imu_gx_p50,  
IDX_imu_gx_p75,  
IDX_imu_gx_p100,
```

```
IDX_imu_gy_first,  
IDX_imu_gy_last,  
IDX_imu_gy_mean,  
IDX_imu_gy_min,  
IDX_imu_gy_max,  
IDX_imu_gy_std,  
IDX_imu_gy_range,  
IDX_imu_gy_delta,  
IDX_imu_gy_abs_change_sum,  
IDX_imu_gy_slope,  
IDX_imu_gy_p0,  
IDX_imu_gy_p25,  
IDX_imu_gy_p50,  
IDX_imu_gy_p75,  
IDX_imu_gy_p100,
```

```
IDX_imu_gz_first,  
IDX_imu_gz_last,  
IDX_imu_gz_mean,  
IDX_imu_gz_min,  
IDX_imu_gz_max,  
IDX_imu_gz_std,  
IDX_imu_gz_range,  
IDX_imu_gz_delta,  
IDX_imu_gz_abs_change_sum,  
IDX_imu_gz_slope,  
IDX_imu_gz_p0,  
IDX_imu_gz_p25,  
IDX_imu_gz_p50,  
IDX_imu_gz_p75,
```

```
IDX_imu_gz_p100,

IDX_imu_mx_first,
IDX_imu_mx_last,
IDX_imu_mx_mean,
IDX_imu_mx_min,
IDX_imu_mx_max,
IDX_imu_mx_std,
IDX_imu_mx_range,
IDX_imu_mx_delta,
IDX_imu_mx_abs_change_sum,
IDX_imu_mx_slope,
IDX_imu_mx_p0,
IDX_imu_mx_p25,
IDX_imu_mx_p50,
IDX_imu_mx_p75,
IDX_imu_mx_p100,

IDX_imu_my_first,
IDX_imu_my_last,
IDX_imu_my_mean,
IDX_imu_my_min,
IDX_imu_my_max,
IDX_imu_my_std,
IDX_imu_my_range,
IDX_imu_my_delta,
IDX_imu_my_abs_change_sum,
IDX_imu_my_slope,
IDX_imu_my_p0,
IDX_imu_my_p25,
IDX_imu_my_p50,
IDX_imu_my_p75,
IDX_imu_my_p100,

IDX_imu_mz_first,
IDX_imu_mz_last,
IDX_imu_mz_mean,
IDX_imu_mz_min,
IDX_imu_mz_max,
```

```
IDX_imu_mz_std,  
IDX_imu_mz_range,  
IDX_imu_mz_delta,  
IDX_imu_mz_abs_change_sum,  
IDX_imu_mz_slope,  
IDX_imu_mz_p0,  
IDX_imu_mz_p25,  
IDX_imu_mz_p50,  
IDX_imu_mz_p75,  
IDX_imu_mz_p100,  
  
IDX_contact_p_first,  
IDX_contact_p_last,  
IDX_contact_p_mean,  
IDX_contact_p_min,  
IDX_contact_p_max,  
IDX_contact_p_std,  
IDX_contact_p_range,  
IDX_contact_p_delta,  
IDX_contact_p_abs_change_sum,  
IDX_contact_p_slope,  
IDX_contact_p_p0,  
IDX_contact_p_p25,  
IDX_contact_p_p50,  
IDX_contact_p_p75,  
IDX_contact_p_p100,  
  
IDX_contact_i_first,  
IDX_contact_i_last,  
IDX_contact_i_mean,  
IDX_contact_i_min,  
IDX_contact_i_max,  
IDX_contact_i_std,  
IDX_contact_i_range,  
IDX_contact_i_delta,  
IDX_contact_i_abs_change_sum,  
IDX_contact_i_slope,  
IDX_contact_i_p0,  
IDX_contact_i_p25,
```



```
IDX_contact_i_p50,  
IDX_contact_i_p75,  
IDX_contact_i_p100,  
  
IDX_contact_m_first,  
IDX_contact_m_last,  
IDX_contact_m_mean,  
IDX_contact_m_min,  
IDX_contact_m_max,  
IDX_contact_m_std,  
IDX_contact_m_range,  
IDX_contact_m_delta,  
IDX_contact_m_abs_change_sum,  
IDX_contact_m_slope,  
IDX_contact_m_p0,  
IDX_contact_m_p25,  
IDX_contact_m_p50,  
IDX_contact_m_p75,  
IDX_contact_m_p100,  
  
IDX_contact_um_first,  
IDX_contact_um_last,  
IDX_contact_um_mean,  
IDX_contact_um_min,  
IDX_contact_um_max,  
IDX_contact_um_std,  
IDX_contact_um_range,  
IDX_contact_um_delta,  
IDX_contact_um_abs_change_sum,  
IDX_contact_um_slope,  
IDX_contact_um_p0,  
IDX_contact_um_p25,  
IDX_contact_um_p50,  
IDX_contact_um_p75,  
IDX_contact_um_p100,  
  
IDX_accel_mag_mean,  
IDX_accel_mag_max,  
IDX_accel_mag_std,
```

```

    IDX_accel_mag_abs_change_sum,
    IDX_gyro_mag_mean,
    IDX_gyro_mag_max,
    IDX_gyro_mag_std,
    IDX_gyro_mag_abs_change_sum,
    IDX_mag_mag_mean,
    IDX_mag_mag_max,
    IDX_mag_mag_std,
    IDX_mag_mag_abs_change_sum
};

// ----- Error Handler -----
void fatalError(const char* msg) {
    Serial.println(msg); // Print the error
    while (1) { // Stop forever
        delay(100); // Small delay while stopped
    }
}

// ----- Small Math Helpers -----
float safeDivide(float a, float b) {
    if (fabs(b) < 0.000001f) return 0.0f; // Avoid divide-by-zero
    return a / b; // Normal division
}

float getMin(const float* arr, int n) {
    float v = arr[0]; // Start with the first value
    for (int i = 1; i < n; i++) if (arr[i] < v) v = arr[i]; // Keep the
    smallest value
    return v; // Return the minimum
}

float getMax(const float* arr, int n) {
    float v = arr[0]; // Start with the first value
    for (int i = 1; i < n; i++) if (arr[i] > v) v = arr[i]; // Keep the
    largest value
    return v; // Return the maximum
}

```

```

float getMean(const float* arr, int n) {
    float sum = 0.0f; // Running sum
    for (int i = 0; i < n; i++) sum += arr[i]; // Add all values
    return safeDivide(sum, (float)n); // Return average
}

float getStd(const float* arr, int n, float mean) {
    float sum = 0.0f; // Sum of squared differences
    for (int i = 0; i < n; i++) {
        float d = arr[i] - mean; // Difference from the mean
        sum += d * d; // Square and add
    }
    return sqrtf(safeDivide(sum, (float)n)); // Return standard
deviation
}

float getAbsChangeSum(const float* arr, int n) {
    float sum = 0.0f; // Running total of absolute changes
    for (int i = 1; i < n; i++) sum += fabs(arr[i] - arr[i - 1]); //
Add change from point to point
    return sum; // Return total change
}

float lerp(float a, float b, float t) {
    return a + (b - a) * t; // Linear interpolation between two values
}

float sampleAtFraction(const float* arr, int n, float fraction) {
    if (n <= 1) return arr[0]; // If only one sample exists, return it
    float pos = fraction * (float)(n - 1); // Convert fraction to array
position
    int left = (int)floor(pos); // Left index
    int right = (int)ceil(pos); // Right index
    if (left == right) return arr[left]; // If exact point, return it
    float t = pos - (float)left; // Fraction between left and right
    return lerp(arr[left], arr[right], t); // Interpolate the value
}

// ----- Feature Writer -----

```

```

// This writes the 15 stats used for one sensor channel
void writeChannelFeatures(
    float* feat, // Output feature vector
    int startIdx, // Where this channel starts in the vector
    const float* arr, // Input data for one channel
    int n, // Number of frames
    float duration_s // Sequence duration
) {
    float first = arr[0]; // First sample
    float last = arr[n - 1]; // Last sample
    float mean = getMean(arr, n); // Mean value
    float minv = getMin(arr, n); // Minimum value
    float maxv = getMax(arr, n); // Maximum value
    float stdv = getStd(arr, n, mean); // Standard deviation
    float range = maxv - minv; // Range
    float delta = last - first; // Change from first to last
    float absChangeSum = getAbsChangeSum(arr, n); // Total
motion/change
    float slope = safeDivide(delta, duration_s); // Average slope
across time
    float p0 = sampleAtFraction(arr, n, 0.00f); // Sample at 0%
    float p25 = sampleAtFraction(arr, n, 0.25f); // Sample at 25%
    float p50 = sampleAtFraction(arr, n, 0.50f); // Sample at 50%
    float p75 = sampleAtFraction(arr, n, 0.75f); // Sample at 75%
    float p100 = sampleAtFraction(arr, n, 1.00f); // Sample at 100%

    feat[startIdx + 0] = first; // Store first
    feat[startIdx + 1] = last; // Store last
    feat[startIdx + 2] = mean; // Store mean
    feat[startIdx + 3] = minv; // Store min
    feat[startIdx + 4] = maxv; // Store max
    feat[startIdx + 5] = stdv; // Store std
    feat[startIdx + 6] = range; // Store range
    feat[startIdx + 7] = delta; // Store delta
    feat[startIdx + 8] = absChangeSum; // Store abs change sum
    feat[startIdx + 9] = slope; // Store slope
    feat[startIdx + 10] = p0; // Store p0
    feat[startIdx + 11] = p25; // Store p25
    feat[startIdx + 12] = p50; // Store p50

```

```

    feat[startIdx + 13] = p75; // Store p75
    feat[startIdx + 14] = p100; // Store p100
}

// ----- Sensor Read Wrapper -----
// This reads one raw frame from the ESP32 pins and sensors
RawFrame readRawFrame() {
    RawFrame f; // Create a new frame
    f.timestamp_ms = millis(); // Save current time

    f.hall_thumb = (float)ads1.readADC_SingleEnded(0); // Read thumb
    hall sensor
    f.hall_index = (float)ads2.readADC_SingleEnded(0); // Read index
    hall sensor
    f.hall_middle = (float)ads2.readADC_SingleEnded(1); // Read middle
    hall sensor
    f.hall_ring = (float)ads2.readADC_SingleEnded(2); // Read ring
    hall sensor
    f.hall_pinky = (float)ads2.readADC_SingleEnded(3); // Read pinky
    hall sensor

    imu::Vector<3> accel =
    bno->getVector(Adafruit_BNO055::VECTOR_ACCELEROMETER); // Read accel
    imu::Vector<3> gyro =
    bno->getVector(Adafruit_BNO055::VECTOR_GYROSCOPE); // Read gyro
    imu::Vector<3> mag =
    bno->getVector(Adafruit_BNO055::VECTOR_MAGNETOMETER); // Read
    magnetometer

    f.imu_ax = accel.x(); // Save accel X
    f.imu_ay = accel.y(); // Save accel Y
    f.imu_az = accel.z(); // Save accel Z

    f.imu_gx = gyro.x(); // Save gyro X
    f.imu_gy = gyro.y(); // Save gyro Y
    f.imu_gz = gyro.z(); // Save gyro Z

    f.imu_mx = mag.x(); // Save mag X
    f.imu_my = mag.y(); // Save mag Y

```

```

f.imu_mz = mag.z(); // Save mag Z

f.contact_p = (float)digitalRead(PALM); // Read palm contact
f.contact_i = (float)digitalRead(INDEX); // Read index contact
f.contact_m = (float)digitalRead(MIDDLE); // Read middle contact
f.contact_um = (float)digitalRead(UNDER_MIDDLE); // Read
under-middle contact

return f; // Return the full frame
}

// ----- Sequence To Feature Vector -----
// This converts the raw sequence buffer into the exact 284-feature
model input
bool buildFeatureVectorFromFrames(const RawFrame* frames, int count,
float* feat) {
    if (count < 2) return false; // Need at least 2 frames to compute
time-based features

    memset(feat, 0, sizeof(float) * RFModel::NUM_FEATURES); // Clear
the output vector

    float duration_s = (float)(frames[count - 1].timestamp_ms -
frames[0].timestamp_ms) / 1000.0f; // Sequence duration in seconds
    if (duration_s <= 0.0f) duration_s = 0.000001f; // Prevent
divide-by-zero

    feat[IDX_duration_s] = duration_s; // Store duration feature
    feat[IDX_num_frames] = (float)count; // Store number of frames
feature

    float hall_thumb[MAX_FRAMES]; // Buffer for thumb channel
    float hall_index[MAX_FRAMES]; // Buffer for index channel
    float hall_middle[MAX_FRAMES]; // Buffer for middle channel
    float hall_ring[MAX_FRAMES]; // Buffer for ring channel
    float hall_pinky[MAX_FRAMES]; // Buffer for pinky channel

    float imu_ax[MAX_FRAMES]; // Buffer for accel X
    float imu_ay[MAX_FRAMES]; // Buffer for accel Y

```

```

float imu_az[MAX_FRAMES]; // Buffer for accel Z

float imu_gx[MAX_FRAMES]; // Buffer for gyro X
float imu_gy[MAX_FRAMES]; // Buffer for gyro Y
float imu_gz[MAX_FRAMES]; // Buffer for gyro Z

float imu_mx[MAX_FRAMES]; // Buffer for mag X
float imu_my[MAX_FRAMES]; // Buffer for mag Y
float imu_mz[MAX_FRAMES]; // Buffer for mag Z

float contact_p[MAX_FRAMES]; // Buffer for palm contact
float contact_i[MAX_FRAMES]; // Buffer for index contact
float contact_m[MAX_FRAMES]; // Buffer for middle contact
float contact_um[MAX_FRAMES]; // Buffer for under-middle contact

float accel_mag[MAX_FRAMES]; // Buffer for accel magnitude
float gyro_mag[MAX_FRAMES]; // Buffer for gyro magnitude
float mag_mag[MAX_FRAMES]; // Buffer for mag magnitude

for (int i = 0; i < count; i++) {
    hall_thumb[i] = frames[i].hall_thumb; // Copy thumb data
    hall_index[i] = frames[i].hall_index; // Copy index data
    hall_middle[i] = frames[i].hall_middle; // Copy middle data
    hall_ring[i] = frames[i].hall_ring; // Copy ring data
    hall_pinky[i] = frames[i].hall_pinky; // Copy pinky data

    imu_ax[i] = frames[i].imu_ax; // Copy accel X
    imu_ay[i] = frames[i].imu_ay; // Copy accel Y
    imu_az[i] = frames[i].imu_az; // Copy accel Z

    imu_gx[i] = frames[i].imu_gx; // Copy gyro X
    imu_gy[i] = frames[i].imu_gy; // Copy gyro Y
    imu_gz[i] = frames[i].imu_gz; // Copy gyro Z

    imu_mx[i] = frames[i].imu_mx; // Copy mag X
    imu_my[i] = frames[i].imu_my; // Copy mag Y
    imu_mz[i] = frames[i].imu_mz; // Copy mag Z

    contact_p[i] = frames[i].contact_p; // Copy palm contact

```

```

    contact_i[i] = frames[i].contact_i; // Copy index contact
    contact_m[i] = frames[i].contact_m; // Copy middle contact
    contact_um[i] = frames[i].contact_um; // Copy under-middle
contact

    accel_mag[i] = sqrtf(
        frames[i].imu_ax * frames[i].imu_ax +
        frames[i].imu_ay * frames[i].imu_ay +
        frames[i].imu_az * frames[i].imu_az
    ); // Compute accel magnitude for this frame

    gyro_mag[i] = sqrtf(
        frames[i].imu_gx * frames[i].imu_gx +
        frames[i].imu_gy * frames[i].imu_gy +
        frames[i].imu_gz * frames[i].imu_gz
    ); // Compute gyro magnitude for this frame

    mag_mag[i] = sqrtf(
        frames[i].imu_mx * frames[i].imu_mx +
        frames[i].imu_my * frames[i].imu_my +
        frames[i].imu_mz * frames[i].imu_mz
    ); // Compute magnetometer magnitude for this frame
}

writeChannelFeatures(feat, IDX_hall_thumb_first, hall_thumb, count,
duration_s); // Write thumb features
writeChannelFeatures(feat, IDX_hall_index_first, hall_index, count,
duration_s); // Write index features
writeChannelFeatures(feat, IDX_hall_middle_first, hall_middle,
count, duration_s); // Write middle features
writeChannelFeatures(feat, IDX_hall_ring_first, hall_ring, count,
duration_s); // Write ring features
writeChannelFeatures(feat, IDX_hall_pinky_first, hall_pinky, count,
duration_s); // Write pinky features

writeChannelFeatures(feat, IDX_imu_ax_first, imu_ax, count,
duration_s); // Write accel X features
writeChannelFeatures(feat, IDX_imu_ay_first, imu_ay, count,
duration_s); // Write accel Y features

```



```

    writeChannelFeatures(feats, IDX_imu_az_first, imu_az, count,
duration_s); // Write accel Z features

    writeChannelFeatures(feats, IDX_imu_gx_first, imu_gx, count,
duration_s); // Write gyro X features
    writeChannelFeatures(feats, IDX_imu_gy_first, imu_gy, count,
duration_s); // Write gyro Y features
    writeChannelFeatures(feats, IDX_imu_gz_first, imu_gz, count,
duration_s); // Write gyro Z features

    writeChannelFeatures(feats, IDX_imu_mx_first, imu_mx, count,
duration_s); // Write mag X features
    writeChannelFeatures(feats, IDX_imu_my_first, imu_my, count,
duration_s); // Write mag Y features
    writeChannelFeatures(feats, IDX_imu_mz_first, imu_mz, count,
duration_s); // Write mag Z features

    writeChannelFeatures(feats, IDX_contact_p_first, contact_p, count,
duration_s); // Write palm contact features
    writeChannelFeatures(feats, IDX_contact_i_first, contact_i, count,
duration_s); // Write index contact features
    writeChannelFeatures(feats, IDX_contact_m_first, contact_m, count,
duration_s); // Write middle contact features
    writeChannelFeatures(feats, IDX_contact_um_first, contact_um, count,
duration_s); // Write under-middle contact features

    feats[IDX_accel_mag_mean] = getMean(accel_mag, count); // Accel
magnitude mean
    feats[IDX_accel_mag_max] = getMax(accel_mag, count); // Accel
magnitude max
    feats[IDX_accel_mag_std] = getStd(accel_mag, count,
feats[IDX_accel_mag_mean]); // Accel magnitude std
    feats[IDX_accel_mag_abs_change_sum] = getAbsChangeSum(accel_mag,
count); // Accel magnitude motion sum

    feats[IDX_gyro_mag_mean] = getMean(gyro_mag, count); // Gyro
magnitude mean
    feats[IDX_gyro_mag_max] = getMax(gyro_mag, count); // Gyro magnitude
max

```

```

    feat[IDX_gyro_mag_std] = getStd(gyro_mag, count,
    feat[IDX_gyro_mag_mean]); // Gyro magnitude std
    feat[IDX_gyro_mag_abs_change_sum] = getAbsChangeSum(gyro_mag,
    count); // Gyro magnitude motion sum

    feat[IDX_mag_mag_mean] = getMean(mag_mag, count); // Magnetometer
    magnitude mean
    feat[IDX_mag_mag_max] = getMax(mag_mag, count); // Magnetometer
    magnitude max
    feat[IDX_mag_mag_std] = getStd(mag_mag, count,
    feat[IDX_mag_mag_mean]); // Magnetometer magnitude std
    feat[IDX_mag_mag_abs_change_sum] = getAbsChangeSum(mag_mag, count);
    // Magnetometer magnitude motion sum

    return true; // Feature vector built successfully
}

// ----- Model Call Function -----
// This is the main function that calls the random forest
int runGestureModel(const float* featureVector) {
    return RFModel::predict(featureVector); // Return predicted class
    index
}

// ----- Friendly Prediction Wrapper -----
// This returns the class label string instead of just the index
const char* predictGestureLabel(const float* featureVector) {
    int classIdx = runGestureModel(featureVector); // Get predicted
    class number
    return RFModel::class_name(classIdx); // Convert class number to
    label text
}

// ----- End-To-End Wrapper -----
// This shows how the ESP32 collects a window, extracts features, and
    calls the model
bool collectAndPredictGesture() {
    g_frameCount = 0; // Start a fresh sequence buffer

```

```

    for (int i = 0; i < MAX_FRAMES; i++) {
        g_frames[g_frameCount] = readRawFrame(); // Read one raw sensor
frame
        g_frameCount++; // Count that frame
        delay(SAMPLE_DELAY_MS); // Wait before next sample
    }

    float featureVector[RFModel::NUM_FEATURES]; // Create the
284-feature input buffer

    if (!buildFeatureVectorFromFrames(g_frames, g_frameCount,
featureVector)) { // Try to build features
        Serial.println("Feature build failed"); // Print error if it
fails
        return false; // Stop here
    }

    int classIdx = runGestureModel(featureVector); // Run the random
forest
    const char* label = RFModel::class_name(classIdx); // Convert
result to label text

    Serial.print("Predicted class index: "); // Print label for
debugging
    Serial.println(classIdx); // Print class index

    Serial.print("Predicted class name: "); // Print label heading
    Serial.println(label); // Print class label

    return true; // Prediction succeeded
}

// ----- Setup -----
void setup() {
    Serial.begin(115200); // Start serial output
    delay(2000); // Give serial monitor time to open

    pinMode(PALM, INPUT_PULLDOWN); // Palm contact input
    pinMode(INDEX, INPUT_PULLDOWN); // Index contact input

```

```

    pinMode(MIDDLE, INPUT_PULLDOWN); // Middle contact input
    pinMode(UNDER_MIDDLE, INPUT_PULLDOWN); // Under-middle contact
    input

    Wire.begin(); // Start I2C
    Wire.setClock(400000); // Use fast I2C

    if (!ads1.begin(0x48, &Wire)) fatalError("ERROR: ADS1115 at 0x48
not found"); // Start first ADC
    if (!ads2.begin(0x49, &Wire)) fatalError("ERROR: ADS1115 at 0x49
not found"); // Start second ADC

    ads1.setGain(GAIN_ONE); // Set ADC gain for ads1
    ads2.setGain(GAIN_ONE); // Set ADC gain for ads2

    if (bno28.begin()) {
        bno = &bno28; // Use IMU at 0x28 if found
    } else if (bno29.begin()) {
        bno = &bno29; // Otherwise use IMU at 0x29 if found
    } else {
        fatalError("ERROR: BNO055 not found at 0x28 or 0x29"); // Stop if
no IMU is found
    }

    bno->setExtCrystalUse(true); // Use external crystal for better
stability
    delay(1000); // Give the IMU time to settle

    Serial.println("System ready"); // Print ready message
}

// ----- Main Loop -----
void loop() {
    collectAndPredictGesture(); // Read one gesture window and classify
it
    delay(500); // Small pause before the next full gesture read
}

```


Appendix F - ASL Charts

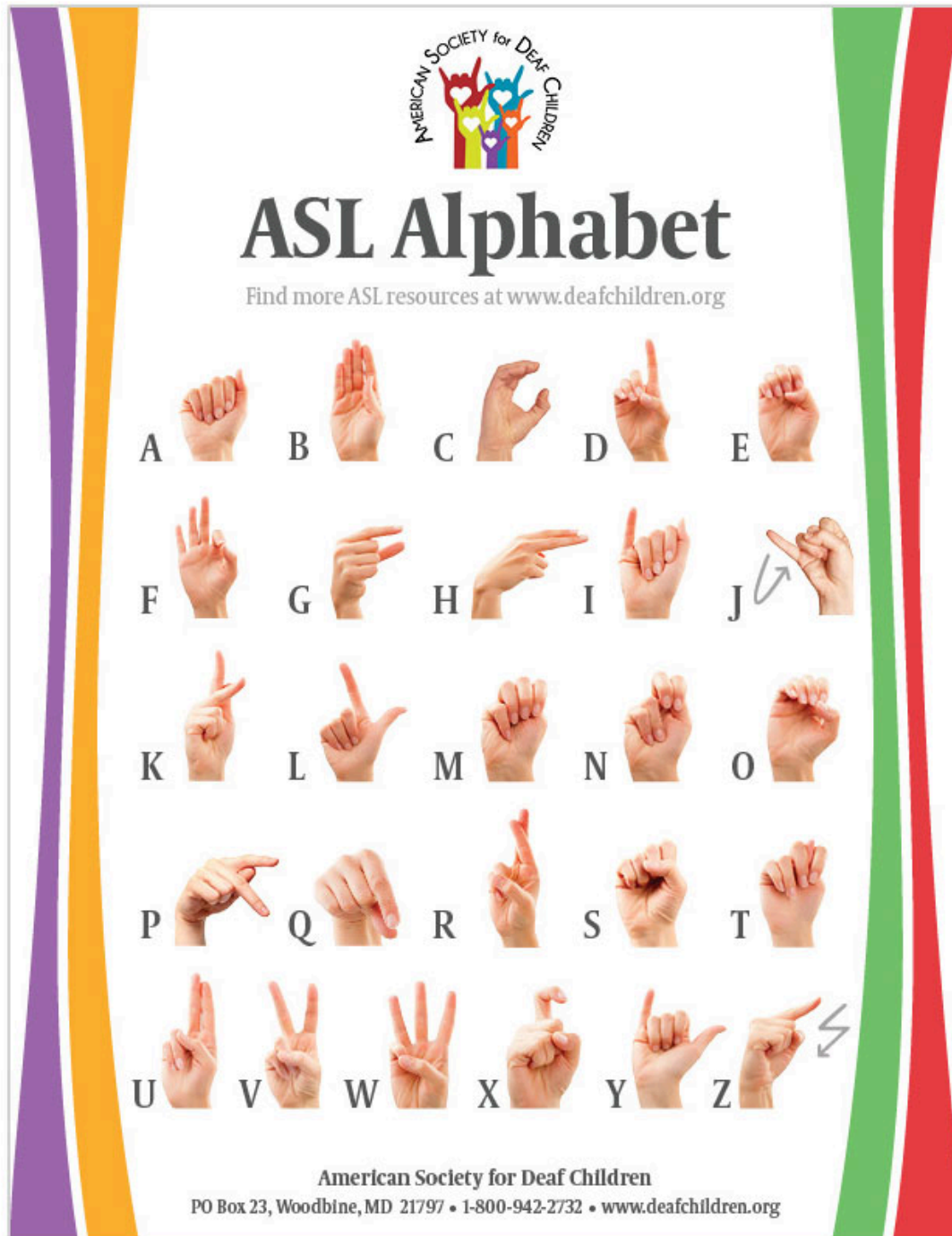


Figure F.1 - ASL Alphabet [4]

AMERICAN SIGN LANGUAGE **numbers** 1-10



WHEN COUNTING, THE NUMBERS ONE THROUGH FIVE ARE SIGNED **PALM IN**.



ALL NUMBERS (EXCEPT ONE THROUGH FIVE) ARE SIGNED **PALM OUT**.

download this poster &  **Signing Savvy**.com
watch sign language videos at YOUR SIGN LANGUAGE RESOURCE

Figure F.2 - ASL Numbers 1-10 [3]